Department Of Aerospace Engineering

Scripting in ANSA using Python

Experiential Learning Project Report

Submitted by,

AKULA UDAY KIRAN	**1RV23AS004**
MOVVA SAI LALITHA DEVI	**1RV23AS030**
SHANTHOSH KV	**1RV23AS053**
TEJAS L	**1RV23AS061**

Under the guidance of

Dr. Benjamin Rohit
Assistant Professor
Dept. of Aerospace Engineering
RV College of Engineering

In partial fulfilment for the award of elective course
of
AEROSPACE STRUCTURES (AS244AI)
Even Semester
2024-2025

RV COLLEGE OF ENGINEERING

BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

Department of Aerospace Engineering

CERTIFICATE

Certified that the experiential learning project work titled **'Scripting in ANSA using Python'** is carried out by **Akula Uday Kiran (1RV23AS004), Movva Sai Lalitha Devi (1RV23AS030), Shanthosh KV (1RV23AS053), Tejas L (1RV23AS061)**, in partial fulfilment for the award of the course, Aerospace Structures (AS244AI) of R.V. College Of Engineering, Bengaluru, affiliated to Visvesvaraya Technological University, Belagavi during the year 2024-2025. It is certified that all corrections/suggestions indicated for the Internal Assessment have been incorporated in the project report deposited in the departmental library. The report has been approved as it satisfies the academic requirements in respect of project work prescribed by the institution.

Signature of Guide	Signature of Head of the Department	Signature of Principal Dr. K N Subramanya

External Viva

Name of Examiners	Signature with Date

RV COLLEGE OF ENGINEERING

BENGALURU-59

(Autonomous Institution Affiliated to VTU, Belagavi)

Department of Aerospace Engineering

DECLARATION

We, **Akula Uday Kiran, Movva Sai Lalitha Devi, Shanthosh KV, Tejas L**, students of fourth semester B.E., department of ASE, RV College of Engineering, Bengaluru, hereby declare that the experiential learning project titled **'Scripting in ANSA using Python'** has been carried out by us and submitted in partial fulfilment for the award of degree of Bachelor of Engineering in Aerospace Engineering during the year 2024-25.

Further we declare that the content of the report has not been submitted previously by anybody for the award of any degree or diploma to any other university.

We also declare that any Intellectual Property Rights generated out of this project carried out at RVCE will be the property of RV College of Engineering, Bengaluru and we will be one of the authors of the same.

Place: Bengaluru
Date:

Name	**Signature**
1. Akula Uday Kiran (1RV23AS004)	
2. Movva Sai Lalitha Devi (1RV23AS030)	
3. Shanthosh KV (1RV23AS053)	
4. Tejas L (1RV23AS061)	

ACKNOWLEDGEMENT

We are indebted to our guide, **Dr. Benjamin Rohit**, Assistant Professor, Dept of Aerospace Engineering for his wholehearted support, suggestions and invaluable advice throughout our project work and also helped in the preparation of this thesis.

Our sincere thanks to **Dr. Supreeth R**, Associate Professor and Head, Department of Aerospace Engineering, RVCE for his support and encouragement.

We express sincere gratitude to our beloved Principal, **Dr. K. N. Subramanya** for his appreciation towards this project work.

We thank all the teaching staff and technical staff of Aerospace Engineering department, RVCE for their help and providing access to simulation software and laboratory facilities.

We are grateful to the management of RV College of Engineering for providing the necessary infrastructure and resources to complete this project successfully.

Lastly, we take this opportunity to thank our family members and friends who provided all the backup support throughout the project work.

Contents

List of Tables

List of Figures

Part I

Computational Meshing Fundamentals

Chapter 1

Introduction

The field of computational engineering has revolutionized the way we approach complex engineering problems. At the heart of most numerical simulations lies the fundamental process of meshing - the discretization of continuous domains into finite elements that can be computationally analyzed. This report provides a comprehensive examination of meshing principles, methodologies, and applications in modern computational analysis.

Meshing serves as the bridge between the continuous mathematical models that describe physical phenomena and the discrete computational algorithms that solve them. Without proper meshing techniques, even the most sophisticated numerical methods would be rendered ineffective. The quality of a mesh directly impacts the accuracy, stability, and computational efficiency of any numerical simulation.

In recent years, the demand for high-fidelity simulations has grown exponentially across industries ranging from aerospace and automotive to biomedical and environmental engineering. This has led to significant advances in meshing algorithms, element types, and quality assessment techniques. Understanding these developments is crucial for engineers and researchers working in computational analysis.

Chapter 2

Understanding Meshing: The Foundation of Numerical Simulations

This chapter provides an exhaustive and highly detailed exposition of meshing, a foundational process in computational simulations across diverse engineering and scientific disciplines, including Finite Element Analysis (FEA), Computational Fluid Dynamics (CFD), and various other numerical methods. It delves deeply into the theoretical underpinnings, practical implications, and advanced considerations of mesh generation. Topics covered include the precise definition, overarching purpose, classification of mesh types, an in-depth analysis of element characteristics, rigorous quality assessment metrics, and a comprehensive breakdown of the meshing workflow, encompassing geometry preparation, algorithmic approaches, and post-meshing validation. Special attention is given to the interplay between mesh characteristics and simulation accuracy, convergence, and computational efficiency.

2.1 Introduction to Computational Discretization

In the contemporary landscape of engineering and scientific research, the analytical solution of partial differential equations (PDEs) governing complex physical phenomena (e.g., fluid flow, heat transfer, structural deformation, electromagnetic fields) is often intractable, if not impossible, due to irregular geometries, non-linear material properties, or intricate boundary conditions. To circumvent this analytical impasse, numerical methods have emerged as indispensable tools, enabling the approximation of solutions to these complex problems. A pivotal and often computationally intensive prerequisite for nearly all numerical simulations is the process of **meshing**, also frequently referred to as **grid generation**.

At its core, meshing represents the fundamental step of *discretization*. It transforms a continuous physical domain, which intrinsically possesses an infinite number of degrees of freedom, into a finite collection of smaller, geometrically simpler, and interconnected sub-domains known as **elements** or **cells**. This collection of elements, along with their shared corner points called **nodes** or **vertices**, constitutes the **computational mesh** or **grid**. The rationale behind this transformation is to convert an infinitely descriptive problem into a computationally tractable problem with a finite number of unknowns that can be solved efficiently by numerical algorithms.

2.1.1 The Indispensability of Meshing in Numerical Methods

The necessity of meshing permeates virtually every stage and aspect of a numerical simulation workflow. Its importance can be elucidated through several critical functions it performs:

- **Transformation to a Discrete Problem:** Numerical methods, such as the Finite Element Method (FEM), Finite Volume Method (FVM), and Finite Difference Method (FDM), are inherently designed to operate on discrete points or finite volumes. Meshing provides the geometric framework upon which these discrete formulations are applied. For instance, in FEM, the governing equations are formulated and solved piecewise over each element, and then assembled globally.

- **Approximation of Complex Geometries and Physics:** Real-world engineering components and fluid domains often exhibit highly intricate geometries. Meshing allows for the accurate approximation of these complex shapes by tessellating them with simpler geometric primitives. Furthermore, it facilitates the piecewise approximation of continuous field variables (e.g., displacement, pressure, temperature) across the domain, often using polynomial interpolation functions defined within each element.

- **Computational Tractability and Efficiency:** Direct analytical or numerical solutions for continuous systems are typically computationally prohibitive. By reducing the problem to a finite number of elements and nodes, meshing effectively transforms an infinite-dimensional problem into a finite-dimensional algebraic system of equations, which can then be solved iteratively or directly by computers. The computational cost, memory requirements, and solution time are directly influenced by the number of elements and nodes in the mesh.

- **Application of Boundary Conditions and Initial Conditions:** Physical boundary conditions (e.g., prescribed displacements, applied forces, fixed temperatures, fluid inlet/outlet velocities, heat fluxes) and initial conditions (for transient problems) are naturally applied at the nodes, edges, or faces of the mesh elements. The mesh provides the necessary discrete interfaces for these conditions.

- **Accurate Representation of Gradients and Fluxes:** Many physical phenomena are driven by gradients (e.g., heat flux driven by temperature gradient, stress driven by strain gradient). A well-designed mesh is crucial for accurately resolving these gradients, especially in regions of high variation, such as boundary layers in fluid flow or stress concentrations in structural components.

- **Post-Processing and Visualization of Results:** Once the numerical solver has converged, the computed solution variables (e.g., stress tensors, velocity vectors, temperature fields) are typically available at the nodes or element centroids of the mesh. The mesh provides the spatial context for interpolating, visualizing, and analyzing these results, often through contour plots, vector fields, or deformation plots.

2.2 The Concept of Discretization: From Continuum to Discrete

Before diving deeper into mesh types, it‘s crucial to grasp the fundamental concept of discretization. Consider a continuous physical domain, Ω, defined by a set of coordinates (x, y, z). A physical variable, $\phi(x, y, z)$, varies continuously throughout this domain. In a continuous formulation, we are solving for ϕ at every single point in Ω.

Discretization replaces this continuous representation with a discrete one. Instead of an infinite number of points, we select a finite number of points (nodes) and define local approximate functions (basis functions) within each element. The continuous variable ϕ is then approximated within an element e as a linear combination of its nodal values and the basis functions:

where n_e is the number of nodes in element e, $N_i(x, y, z)$ are the shape or basis functions associated with node i, and ϕ_i are the unknown values of the variable at node i. The meshing process essentially defines these elements, their connectivity, and the nodal locations.

2.3 Classification and Characteristics of Mesh Types

Meshes are broadly categorized based on the regularity of their element arrangement and connectivity. Each type presents distinct advantages and disadvantages, making their selection dependent on the specific application, geometric complexity, and computational resources.

2.3.1 Structured Meshes

Structured meshes are characterized by a highly organized, curvilinear grid-like arrangement of elements, where each interior node has a fixed and predictable number of neighbors. This topological regularity often implies a logical I, J, K indexing scheme, similar to a 3D array, which simplifies data storage and access.

- **Definition:** Consist of quadrilateral (2D) or hexahedral (3D) elements arranged in a regular, topologically Cartesian pattern, even if geometrically distorted to conform to boundaries. The connectivity between elements is implicit and uniform.

- **Advantages:**

  - **High Accuracy and Resolution:** For problems with dominant flow directions (e.g., boundary layers in CFD), structured meshes, particularly those highly refined in one direction, can provide superior resolution and accuracy with fewer elements compared to unstructured meshes.

  - **Computational Efficiency:** The regular connectivity allows for highly optimized algorithms, efficient memory access patterns (e.g., cache coherence), and often faster convergence for certain solvers (e.g., Finite Difference Method, some Finite Volume solvers). This is particularly true for explicit schemes.

  - **Simpler Data Structures:** The implicit connectivity reduces the need for explicit storage of neighbor lists, simplifying coding and memory management.

  - **Smoother Transitions:** Element size transitions tend to be smoother, which can improve solution stability.

- **Disadvantages:**

  - **Geometric Restrictions:** Generating structured meshes for complex, arbitrary geometries is extremely challenging and often impossible. Multi-block structured meshing (dividing the domain into simpler blocks that can be structured meshed) is a common technique, but still complex.

  - **Time-Consuming Generation:** For anything beyond very simple geometries, manual or semi-automated structured mesh generation can be a laborious and time-consuming process.

  - **Difficult Local Refinement:** Localized refinement (e.g., around a small hole or sharp corner) often requires remeshing large portions of the domain or employing complex grid stretching techniques.

- **Common Elements:** Quadrilaterals (2D), Hexahedra (3D).

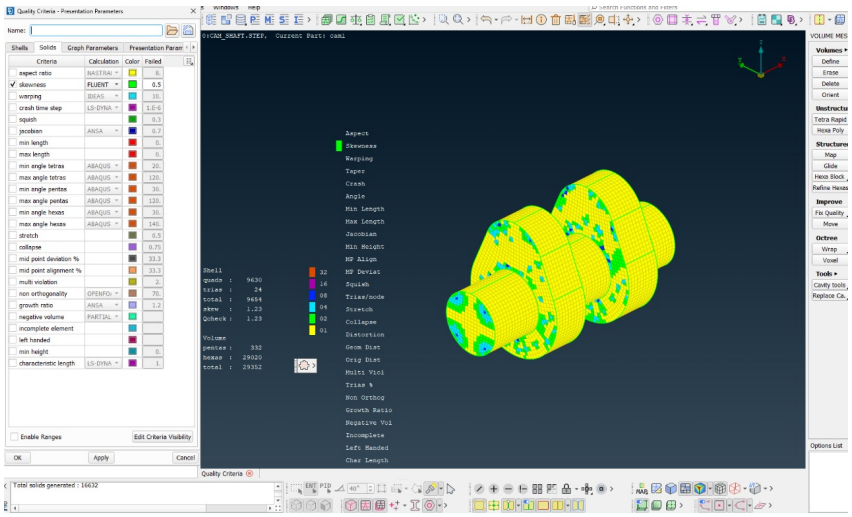

Figure 2.1: Conceptual Illustration of a Structured Mesh. Note the regular, grid-like arrangement and implicit connectivity.

2.3.2 Unstructured Meshes

Unstructured meshes offer unparalleled flexibility in conforming to complex geometries by allowing arbitrary arrangements of elements. Connectivity between elements must be explicitly stored.

- **Definition:** Composed of elements (typically triangles in 2D, tetrahedra in 3D, or a combination of types) with irregular connectivity. Each node can have a variable number of neighbors, and the element sizing can vary significantly across the domain.
- **Advantages:**
  - **Geometric Versatility:** Unstructured meshing algorithms can handle highly complex geometries with arbitrary features (holes, fillets, sharp corners) with relative ease. This is their most significant advantage.
  - **Automated Generation:** Advanced algorithms allow for largely automated mesh generation, significantly reducing meshing time for complex models.
  - **Local Refinement Capability:** Elements can be refined locally in regions of interest (e.g., high gradients, areas of interest) without affecting the entire mesh, allowing for efficient use of computational resources. This is crucial for adaptive mesh refinement (AMR).
  - **Solution Adaptivity:** Meshes can be automatically adapted (refined or coarsened) during the simulation based on solution gradients or error indicators, further optimizing accuracy and efficiency.
- **Disadvantages:**
  - **Complex Data Structures:** Explicit storage of element-to-node and element-to-element connectivity requires more complex data structures and memory, potentially leading to higher memory footprint.
  - **Computational Cost:** While element count can be optimized by local refinement, the irregular nature often leads to less optimized memory access and can result in slower convergence for some solvers compared to structured meshes with similar element counts.
  - **Potential for Poor Quality Elements:** Automated unstructured meshing can sometimes generate elements with poor quality (e.g., high skewness, high aspect ratio) if not carefully controlled, which can adversely affect solution accuracy and stability.

- **Boundary Layer Resolution:** Resolving thin boundary layers (critical in CFD) accurately with purely tetrahedral meshes often requires a very high number of elements, which can be computationally expensive. This often leads to hybrid approaches.

2.3.3 Hybrid Meshes

Hybrid meshes represent a pragmatic approach that combines the strengths of both structured and unstructured meshing strategies within a single computational domain. This is particularly prevalent in CFD.

- **Definition:** Consist of a combination of different element types. A common strategy involves using structured-like elements (e.g., hexahedra or quadrilateral cells) in critical regions where flow is well-aligned or gradients are high (e.g., boundary layers near walls) and unstructured elements (e.g., tetrahedra, triangles) in other regions where geometry is complex or flow is less critical.
- **Advantages:**
  - **Optimized Resolution:** Enables highly accurate resolution of phenomena like boundary layers with prismatic (wedge) elements, which are more efficient than tetrahedra for this purpose, while maintaining geometric flexibility in the bulk domain with tetrahedra.
  - **Reduced Element Count:** Often leads to a lower total element count for a given level of accuracy compared to purely unstructured meshes, especially for problems dominated by boundary layers.
  - **Improved Convergence:** Can lead to better convergence characteristics than purely unstructured meshes due to the higher quality and directional alignment of elements in critical regions.
- **Disadvantages:**
  - **Meshing Complexity:** Generating high-quality hybrid meshes is more complex than generating purely structured or unstructured meshes, requiring specialized algorithms and greater user expertise.
  - **Interface Management:** Careful management of the interfaces between different mesh types is crucial to ensure smooth transitions and accurate solution transfer.
- **Common Combinations:**
  - **2D:** Quadrilaterals in boundary layers, triangles elsewhere.
  - **3D:** Hexahedra/Prisms (wedges) in boundary layers, tetrahedra in the core volume. Pyramids are often used as transitional elements between hexahedra and tetrahedra.

2.3.4 Specialized Mesh Types

Beyond the primary classifications, specialized mesh types exist for particular applications:

- **Body-Fitted Meshes:** Meshes whose boundaries conform exactly to the physical boundaries of the computational domain. Most FEA and CFD meshes are body-fitted.
- **Cartesian Meshes (with cut cells):** A uniform, axis-aligned Cartesian grid that overlays the geometry. Cells that intersect the geometry are "cut," and special treatment is applied to these cut cells to enforce boundary conditions. Often used in immersed boundary methods or for very fast initial grid generation.

- **Overset/Chimera Meshes:** Multiple overlapping, independently generated meshes are used to cover a complex domain, often with moving bodies. Information is interpolated between overlapping regions. Useful for relative motion problems.
- **Adaptive Meshes (AMR):** Meshes that are dynamically refined or coarsened during the simulation based on solution gradients, error estimators, or other criteria, allowing for optimal allocation of computational resources.
- **Polyhedral Meshes:** A relatively newer development, especially in CFD, where elements can have an arbitrary number of faces (typically 10-14 faces). They offer good numerical stability and can efficiently fill space, often requiring fewer elements than tetrahedral meshes for comparable accuracy.

Chapter 3

Mesh Elements: The Building Blocks of Discretization

The choice of element type is fundamental and depends on the dimensionality of the problem, the specific numerical method (e.g., FEM vs. FVM), and the desired accuracy. Elements are broadly classified by their dimensionality and number of nodes.

3.1 Element Order: Linear vs. Higher-Order

Elements are often classified by their polynomial order, which dictates how the solution variable is interpolated within the element.

- **Linear Elements (First-Order):** These elements use linear interpolation functions (shape functions) to approximate the field variable between their corner nodes. For example, a 3-node triangle or a 4-node quadrilateral.
  - **Advantages:** Simpler to implement, computationally less expensive per element, widely used.
  - **Disadvantages:** Can exhibit "shear locking" in bending-dominated problems in FEA, requires more elements for complex curves, and provides constant (or piecewise constant) derivatives within elements, leading to less accurate stress/flux calculations.
- **Higher-Order Elements (Quadratic, Cubic, etc.):** These elements include additional nodes (e.g., mid-side nodes, face nodes, interior nodes) and use higher-order polynomial interpolation functions. For example, a 6-node triangle (T6) or an 8-node quadrilateral (Q8) in 2D, or a 10-node tetrahedron (T10) or a 20-node hexahedron (H20) in 3D.
  - **Advantages:** Significantly improve accuracy, especially for capturing non-linear variations, bending, and stress concentrations. Can represent curved boundaries more accurately. Requires fewer elements than linear elements for the same level of accuracy in many cases.
  - **Disadvantages:** More complex to implement, higher computational cost per element (more degrees of freedom), and can sometimes be more sensitive to element distortion.

3.2 2D Elements

These are used for 2D planar problems (e.g., plane stress, plane strain, axisymmetric problems) or for meshing surfaces in 3D.

- **Triangle (T3, T6, T10, etc.):**
  - **T3 (3-node linear):** Simplest 2D element. Fast to generate, good for complex boundaries but can be "stiff" in FEA.
  - **T6 (6-node quadratic):** Includes 3 corner nodes and 3 mid-side nodes. Provides much better accuracy for bending and curved boundaries.
- **Quadrilateral (Q4, Q8, Q9, etc.):**
  - **Q4 (4-node linear):** Often preferred over triangles for FEA due to better performance in many scenarios. Can suffer from "hourglassing" in certain dynamic problems.
  - **Q8 (8-node quadratic):** Includes 4 corner nodes and 4 mid-side nodes. Also known as serendipity element. Excellent for structural analysis.
  - **Q9 (9-node quadratic):** Includes 4 corner nodes, 4 mid-side nodes, and 1 central node. Offers slightly improved performance over Q8 for some cases.

3.3 3D Elements

These are the most common elements for full 3D analysis of solids and structures.

- **Tetrahedron (Tet4, Tet10, etc.):**
  - **Tet4 (4-node linear):** Simplest 3D element, excellent for complex geometries, automatic meshing. Can be overly stiff for bending problems.
  - **Tet10 (10-node quadratic):** Significantly improved accuracy, especially for stress concentrations and curved boundaries.
- **Hexahedron (Hex8, Hex20, Hex27, etc.):**
  - **Hex8 (8-node linear):** Brick-like element, computationally efficient, excellent for structural analysis but difficult to generate for complex geometries.
  - **Hex20 (20-node quadratic):** High accuracy, particularly good for stress analysis and contact problems.
- **Prism/Wedge (Penta6, Penta15, etc.):**
  - Used for transitioning between different mesh types, particularly useful in boundary layer meshing for CFD.
- **Pyramid (Pyra5, Pyra13, etc.):**
  - Transitional elements between tetrahedral and hexahedral regions in hybrid meshes.

Chapter 4

Mesh Quality Assessment and Optimization

The quality of a computational mesh is perhaps the most critical factor determining the success of a numerical simulation. Poor mesh quality can lead to convergence difficulties, numerical instabilities, and inaccurate results. This chapter explores the various metrics used to assess mesh quality and techniques for optimization.

4.1 Geometric Quality Metrics

Several geometric parameters are used to evaluate element quality:

4.1.1 Aspect Ratio

The aspect ratio measures the deviation of an element from an ideal shape. For triangular and tetrahedral elements, it is typically defined as the ratio of the longest edge to the shortest altitude. High aspect ratios can lead to numerical problems, particularly in explicit time integration schemes.

4.1.2 Skewness

Skewness measures how close an element is to its ideal shape (equilateral triangle, regular tetrahedron, etc.). Elements with high skewness can significantly degrade solution accuracy and convergence.

4.1.3 Jacobian

The Jacobian determinant relates the element coordinate system to the global coordinate system. Negative Jacobians indicate inverted elements, which are unacceptable in most numerical methods.

4.1.4 Warpage

For quadrilateral and hexahedral elements, warpage measures the deviation from planarity. Highly warped elements can lead to accuracy issues.

4.2 Solution-Based Quality Assessment

Beyond geometric metrics, solution-based quality assessment examines how well the mesh resolves the physics of the problem:

4.2.1 Solution Gradients

Regions with high solution gradients require finer meshes for accurate resolution. Adaptive mesh refinement techniques use gradient-based error estimators to guide mesh optimization.

4.2.2 Convergence Studies

Mesh convergence studies involve systematically refining the mesh and monitoring the convergence of solution variables. This is essential for establishing mesh independence of the solution.

4.3 Mesh Optimization Techniques

Several techniques can be employed to improve mesh quality:

4.3.1 Smoothing

Mesh smoothing algorithms adjust node positions to improve element quality while maintaining the overall mesh topology. Laplacian smoothing is one of the most common approaches.

4.3.2 Refinement and Coarsening

Local mesh refinement increases element density in regions requiring higher resolution, while coarsening reduces density where high resolution is not needed.

4.3.3 Topology Optimization

Advanced techniques can modify the mesh topology (connectivity) to improve overall quality, such as edge swapping in triangular meshes.

Part II

Automation and Scripting in Computational Meshing

Chapter 5

Automation in Pre-Processing Software

5.1 Background

The complexity and time-intensive nature of mesh generation for engineering simulations has driven the development of automation techniques. Modern pre-processing software packages provide scripting interfaces that enable the automation of repetitive meshing tasks, significantly reducing human effort and improving consistency in mesh quality.

5.1.1 Python in Engineering Automation

Python has emerged as a powerful, high-level programming language particularly well-suited for automation tasks in engineering applications. Its ease of syntax, comprehensive standard libraries, and robust third-party ecosystem make it an ideal choice for scripting in computational analysis workflows. The language‘s interpreted nature allows for rapid prototyping and iterative development, while its extensive numerical computing libraries (NumPy, SciPy, matplotlib) provide seamless integration with computational workflows.

Key advantages of Python for meshing automation include:

- **Readability and Maintainability:** Python‘s clear syntax structure makes scripts easy to understand, modify, and maintain by different team members.
- **Cross-Platform Compatibility:** Python scripts can run across different operating systems without modification.
- **Integration Capabilities:** Python can interface with various CAE software packages through their respective APIs.
- **Data Processing:** Excellent capabilities for handling geometry data, mesh metrics, and simulation results.

5.1.2 ANSA Pre-Processing Software

ANSA is a comprehensive pre-processing software developed by BETA CAE Systems, specifically designed for Computer-Aided Engineering (CAE) modeling applications. The software supports a

wide range of functionalities essential for finite element analysis preparation, including geometry clean-up, surface and volume meshing, and model setup for various solvers including Abaqus, ANSYS, NASTRAN, LS-DYNA, and others.

ANSA‘s key capabilities relevant to automated meshing include:

- **Geometry Import and Clean-up:** Supports multiple CAD formats with robust geometry repair and preparation tools.
- **Advanced Meshing Algorithms:** Provides both structured and unstructured meshing capabilities with sophisticated quality control.
- **Multi-Solver Support:** Can generate meshes compatible with various finite element solvers.
- **Python API:** Comprehensive scripting interface that allows complete automation of the meshing workflow.
- **Batch Processing:** Capability to process multiple models or configurations in automated sequences.

The ANSA Python API provides access to all major functionalities through well-structured modules and functions, enabling users to create sophisticated automation scripts that can handle complex meshing scenarios with minimal manual intervention.

Chapter 6

Meshing Automation Strategy and Implementation

6.1 Automation Strategy Development

The development of an effective meshing automation strategy requires careful consideration of the specific requirements, geometry characteristics, and analysis objectives. A systematic approach ensures that the automated workflow produces consistent, high-quality results while minimizing manual intervention.

6.1.1 Primary Objectives

The automation strategy is built around three fundamental objectives that form the core of any effective meshing workflow:

- **Automatic Geometry Import:** Seamless importation of CAD geometry from various formats (STEP, IGES, Parasolid, etc.) with automated geometry validation and repair where necessary. This includes handling of assembly structures, part identification, and geometric feature recognition.
- **High-Quality Mesh Generation:** Implementation of robust meshing algorithms that consistently produce meshes meeting specified quality criteria. This involves setting appropriate element sizes, controlling aspect ratios, ensuring smooth transitions, and managing geometric complexity.
- **Solver-Compatible Export:** Automated export of the meshed model to the target solver format with proper material assignments, boundary condition preparation frameworks, and quality validation reports.

6.1.2 Workflow Architecture

The automation workflow follows a modular architecture that allows for flexibility and maintainability:

1. **Pre-Processing Phase:** Geometry import, validation, and preparation
2. **Meshing Phase:** Surface mesh generation followed by volume mesh creation
3. **Quality Assessment Phase:** Automated quality checking and optimization
4. **Post-Processing Phase:** Export preparation and file generation
5. **Validation Phase:** Final checks and reporting

6.2 Phase 1 Implementation: Basic Automation

The initial phase of automation focuses on establishing the fundamental meshing workflow for specific component types. This implementation demonstrates the core concepts while maintaining simplicity and reliability.

6.2.1 Phase 1 Methodology

The Phase 1 script targets volume meshing for mechanical components, specifically focusing on lobes and shafts which are common in rotating machinery applications.

The script architecture involves:

- Direct entity retrieval using property-based identification
- Sequential surface mesh generation using mapped block techniques
- Volume mesh creation with controlled parameters
- Quality-based remeshing for optimization

6.3 Phase 2 Implementation: Enhanced Automation

Building upon the foundation established in Phase 1, the Phase 2 implementation introduces significant enhancements in terms of robustness, flexibility, and functionality. This advanced version incorporates object-oriented design principles, comprehensive error handling, and multi-scale meshing capabilities.

6.3.1 Phase 2 Enhancements

The Phase 2 script represents a substantial evolution from the initial implementation, incorporating several critical improvements:

- **Object-Oriented Architecture:** The functionality is encapsulated within a `MeshProcessor` class, promoting code reusability, maintainability, and extensibility. This design pattern allows for easy integration into larger automation frameworks.
- **Multi-Scale Processing:** Support for batch processing of multiple mesh sizes (e.g., 3mm, 5mm, 7mm) enables convergence studies and mesh sensitivity analyses. This is crucial for validation of numerical results and optimization of computational efficiency.
- **Advanced Logging System:** Implementation of comprehensive logging captures detailed information about each processing step, including timing data, success/failure rates, and quality metrics. This facilitates debugging, process monitoring, and performance optimization.
- **Robust Error Handling:** Sophisticated exception handling mechanisms ensure that the script can recover from errors and continue processing subsequent configurations. This is essential for batch processing scenarios.
- **Automated Export Functionality:** The script automatically exports meshed models to solver-compatible formats with unique identifiers, ensuring traceability and preventing file conflicts in batch processing environments.
- **Quality Reporting:** Automated generation of mesh quality reports provides immediate feedback on mesh characteristics and identifies potential issues before solver execution.

6.3.2 Phase 2 Workflow Architecture

The enhanced workflow follows a systematic approach designed to handle complex meshing scenarios:

1. **Initialization:** Setup of logging systems, parameter validation, and workspace preparation
2. **Geometry Processing:** CAD import with validation and geometric feature identification
3. **Multi-Scale Meshing:** Iterative processing of different mesh sizes with individual quality control
4. **Quality Assessment:** Comprehensive evaluation of mesh quality metrics for each configuration
5. **Export and Documentation:** Automated file generation with detailed process documentation
6. **Summary Reporting:** Consolidated results with performance metrics and recommendations

The Phase 2 implementation maintains backward compatibility with Phase 1 workflows while providing the flexibility to handle more complex scenarios and requirements. The modular design allows for easy customization and extension for specific application domains or solver requirements.

6.4 Log File Overview

The log file records five runs, each attempting to generate a mesh with a specific element size (7 mm three times, 3 mm once, 5 mm once). Each run follows a consistent sequence:

1. Initialize the script and specify the mesh size.
2. Load the STEP file and collect `PSHELL` (shell elements) and `FACE` (surface geometry) entities.
3. Generate mapped block meshes for lobes and shaft.
4. Collect `PSOLID` (solid elements) and `VOLUME` entities, then remesh volumes for `lob1`, `lob2`, and `shaft1`.
5. Attempt to export the mesh, which fails with an `AttributeError`.
6. Log completion and list failed mesh sizes.

6.5 Detailed Analysis of Runs

6.5.1 Run 2: Mesh Size 3

```
2025-06-15 15:08:22,558 - INFO - Script started.
2025-06-15 15:08:22,559 - INFO - Processing mesh size: 3
2025-06-15 15:08:23,735 - INFO - STEP file opened: C:\Users\kvsha\Downloads\Assem1.
    STEP
2025-06-15 15:08:23,736 - INFO - Meshing parameters set (absolute length = 3).
2025-06-15 15:08:25,695 - INFO - Mapped block mesh generated for lobes.
2025-06-15 15:08:26,536 - INFO - Mapped block mesh generated for shaft.
2025-06-15 15:08:26,719 - INFO - Remeshed volume for lob1.
2025-06-15 15:08:26,868 - INFO - Remeshed volume for lob2.
2025-06-15 15:08:27,094 - INFO - Remeshed volume for shaft1.
2025-06-15 15:08:27,096 - ERROR - Error processing mesh size 3: module ‘base‘ has no
    attribute ‘Export‘
```

Listing 6.1: Run 2: Mesh Size 3

Using a finer 3 mm mesh, surface meshing takes longer (2 seconds for lobes, 0.8 seconds for shaft) due to increased element count. Volume meshing remains fast.

6.6 Error Analysis

All runs fail with the same error:

```
Traceback (most recent call last):
  File ‘‘C:/Users/kvsha/Downloads/ansa_mesh_diff_mesh_sizes.py (Run from Script Editor
    )‘‘, line 170, in <module>
  File ‘‘C:/Users/kvsha/Downloads/ansa_mesh_diff_mesh_sizes.py (Run from Script Editor
    )‘‘, line 111, in export_meshed_file
AttributeError: module ‘base‘ has no attribute ‘Export‘
```

Listing 6.2: Export Error

The `AttributeError` indicates the script attempts to call `base.Export`, which does not exist in the `base` module. Possible causes include:

- Incorrect function name (e.g., `Export` may be `Save`, `Write`, or in another module like `base.meshing`).
- Missing or incorrect import statement for the `base` module.
- Incompatibility between the script and the ANSA version.

6.7 Proposed Fixes

To resolve the error:

1. **Consult ANSA API Documentation**: Verify the correct export method (e.g., `base.Save` or `meshing.Export`).
2. **Verify Imports**: Ensure the correct module is imported:

```
import base  % Confirm this module contains the export function

```

3. **Check ANSA Version**: Confirm the script is compatible with the installed ANSA version.

6.8 Conclusion

The log file shows successful attempts to mesh a STEP file but unsuccessful to export. Meshing steps for surface and volume elements complete successfully, but the script fails to export the mesh due to an invalid `base.Export` call.

Line-by-Line Explanation of ANSA Meshing Automation Script

This document provides a detailed, line-by-line explanation of a Python script that automates finite element meshing in ANSA for NASTRAN analysis. .

1. Imports

```
import os
import ANSA
import logging
from ANSA import base, constants, mesh
import uuid
```

Listing 6.3: Imports

Explanation:

- `import os`: Imports the `os` module for file operations like checking file existence and retrieving file sizes.
- `import ANSA`: Imports the `ANSA` module, the Python API for ANSA, a finite element pre-processing software.
- `import logging`: Imports the `logging` module for generating log messages to track script activities.
- `from ANSA import base, constants, mesh`: Imports specific submodules from `ANSA`: `base` for core functions, `constants` for solver constants, and `mesh` for meshing functions.
- `import uuid`: Imports the `uuid` module to generate unique identifiers for output file names.

2. Logging Configuration

```
log_path = r''C:\Users\kvsha\Documents\ANSA_mesh_log.txt''
logging.basicConfig(
    filename=log_path,
    filemode=''w'',
    level=logging.INFO,
    format=''%(asctime)s - %(levelname)s - %(message)s''
)
```

Listing 6.4: Logging Configuration

Explanation:

- `log_path = r''C:\Users\kvsha\Documents\ANSA_mesh_log.txt''`: Defines the log file path using a raw string to handle Windows backslashes.
- `logging.basicConfig(...)`: Configures the logging system:
- `filename=log_path`: Specifies the log file path.
- `filemode=''w''`: Sets write mode, overwriting the log file if it exists.
- `level=logging.INFO`: Sets the logging level to capture `INFO` messages and above.
- `format=''%(asctime)s - %(levelname)s - %(message)s''`: Defines the log format as timestamp, level, and message.

3. Solver Configuration

```
deck = constants.NASTRAN
```

Listing 6.5: Solver Configuration

Explanation:

- `deck = constants.NASTRAN`: Sets the solver format to NASTRAN, ensuring the mesh and output files are NASTRAN-compatible.

4. MeshProcessor Class - Part 1 (Initial Steps)

```
class MeshProcessor:
    def main(self, mesh_size):
        logging.info(''-----------'')
        logging.info(f''Starting comprehensive mesh processing workflow for mesh size {mesh_size} mm.'')
        logging.info(f''Target element size: {mesh_size} (this will determine mesh density and computational requirements)'')

        step_file = r''C:\Users\kvsha\Downloads\Assem1.STEP''
        base.Open(step_file)
        logging.info(f''Successfully opened STEP file: {step_file}'')
        logging.info(''CAD geometry has been loaded into ANSA workspace for processing'')
```

Listing 6.6: MeshProcessor Class - Initial Steps

Explanation:

- `class MeshProcessor:`: Defines a class to encapsulate the meshing workflow.
- `def main(self, mesh_size):`: Defines the `main` method, which takes a `mesh_size` parameter (target element size in mm).
- `logging.info(''------'')`: Logs a separator line for visual separation.
- `logging.info(f''Starting comprehensive mesh processing workflow...'')`: Logs the start of the meshing workflow for the given `mesh_size`.
- `logging.info(f''Target element size: {mesh_size}...'')`: Logs the target element size and its impact on mesh density.
- `step_file = r''C:\Users\kvsha\Downloads\Assem1.STEP''`: Defines the path to the STEP file containing CAD geometry.
- `base.Open(step_file)`: Loads the STEP file into ANSA's workspace.
- `logging.info(f''Successfully opened STEP file: {step_file}'')`: Logs successful opening of the STEP file.
- `logging.info(''CAD geometry has been loaded...'')`: Logs that the CAD geometry is ready for processing.

5. MeshProcessor Class - Part 2 (Shell Element Retrieval and Surface Collection)

```
        lobes = base.GetEntity(deck, ''PSHELL'', 1)
        shaft = base.GetEntity(deck, ''PSHELL'', 2)
        logging.info(''Successfully retrieved PSHELL entities for shell element definition'')
        logging.info(''- PSHELL ID 1 (lobes): Retrieved for complex curved surface meshing'')
```

```
        logging.info(``- PSHELL ID 2 (shaft): Retrieved for cylindrical/prismatic surface meshing``)

        ents1 = base.CollectEntities(deck, lobes, ``FACE``)
        ents2 = base.CollectEntities(deck, shaft, ``FACE``)
        logging.info(``Successfully collected FACE entities for surface meshing operations``)
        logging.info(f``- Lobe faces collected: {len(ents1) if ents1 else 0} surfaces identified for meshing``)
        logging.info(f``- Shaft faces collected: {len(ents2) if ents2 else 0} surfaces identified for meshing``)
```

Listing 6.7: MeshProcessor Class - Shell Element Retrieval

Explanation:

- `lobes = base.GetEntity(deck, ``PSHELL``, 1)`: Retrieves the `PSHELL` entity with ID 1 (lobes) for shell element properties.
- `shaft = base.GetEntity(deck, ``PSHELL``, 2)`: Retrieves the `PSHELL` entity with ID 2 (shaft).
- `logging.info(``Successfully retrieved PSHELL entities...``)`: Logs successful retrieval of `PSHELL` entities.
- `logging.info(``- PSHELL ID 1 (lobes)...``)`: Logs that ID 1 is for lobe meshing.
- `logging.info(``- PSHELL ID 2 (shaft)...``)`: Logs that ID 2 is for shaft meshing.
- `ents1 = base.CollectEntities(deck, lobes, ``FACE``)`: Collects `FACE` entities (surfaces) for the lobes.
- `ents2 = base.CollectEntities(deck, shaft, ``FACE``)`: Collects `FACE` entities for the shaft.
- `logging.info(``Successfully collected FACE entities...``)`: Logs successful collection of `FACE` entities.
- `logging.info(f``- Lobe faces collected: {len(ents1) if ents1 else 0}...``)`: Logs the number of lobe faces collected.
- `logging.info(f``- Shaft faces collected: {len(ents2) if ents2 else 0}...``)`: Logs the number of shaft faces collected.

6. MeshProcessor Class - Part 3 (Meshing Setup and Surface Meshing)

```
        base.SetCurrentMenu(``VOLUME MESH``)
        logging.info(``Switched to VOLUME MESH menu - volume meshing tools are now active``)

        mesh.SetMeshParamTargetLength(``absolute``, mesh_size)
        logging.info(f``Mesh parameters configured for absolute element sizing``)
        logging.info(f``- Target element edge length: {mesh_size} mm``)
        logging.info(f``- This setting will be applied globally to all meshing operations``)
        logging.info(f``- Estimated elements per surface area: ~{(10/mesh_size)**2:.0f} elements per 100 \si{mm^2}``)

        mesh.MapBlock(ents1)
        logging.info(``Successfully generated mapped block mesh for lobe components``)
        logging.info(``- Used structured meshing algorithm for optimal element quality``)
```

```
13          logging.info(‘‘- Quadrilateral shell elements created with consistent
    orientation‘‘)
14
15          mesh.MapBlock(ents2)
16          logging.info(‘‘Successfully generated mapped block mesh for shaft components
    ‘‘)
17          logging.info(‘‘- Structured mesh pattern applied to cylindrical surfaces‘‘)
18          logging.info(‘‘- Element alignment optimized for circumferential and axial
    directions‘‘)
```

Listing 6.8: MeshProcessor Class - Meshing Setup and Surface Meshing

Explanation:

- `base.SetCurrentMenu(‘‘VOLUME MESH‘‘)`: Switches ANSA to the `VOLUME MESH` menu to access volume meshing tools.
- `logging.info(‘‘Switched to VOLUME MESH menu...‘‘)`: Logs the menu switch.
- `mesh.SetMeshParamTargetLength(‘‘absolute‘‘, mesh_size)`: Sets the mesh size to `mesh_size` using absolute sizing.
- `logging.info(f‘‘Mesh parameters configured...‘‘)`: Logs that mesh parameters are configured.
- `logging.info(f‘‘- Target element edge length: {mesh_size} mm‘‘)`: Logs the target element size.
- `logging.info(f‘‘- This setting will be applied globally...‘‘)`: Logs that the setting is global.
- `logging.info(f‘‘- Estimated elements per surface area: ~{(10/mesh_size)**2:.0f} elements per 100 mm$^2$‘‘)` :Logs an estimate of elements per 100 mm^2.
- `mesh.MapBlock(ents1)`: Generates a surface mesh for lobe faces using mapped block meshing.
- `logging.info(‘‘Successfully generated mapped block mesh for lobe components‘‘)`: Logs successful meshing of lobes.
- `logging.info(‘‘- Used structured meshing algorithm...‘‘)`: Logs the use of a structured meshing algorithm.
- `logging.info(‘‘-Quadrilateral shell elements created...‘‘)`: Logs the creation of quadrilateral elements.
- `mesh.MapBlock(ents2)`: Generates a surface mesh for shaft faces.
- `logging.info(‘‘Successfully generated mapped block mesh for shaft components‘‘)`: Logs successful meshing of the shaft.
- `logging.info(‘‘- Structured mesh pattern applied...‘‘)`: Logs the application of a structured mesh pattern.
- `logging.info(‘‘- Element alignment optimized...‘‘)`: Logs optimization of element alignment for the shaft.

7. MeshProcessor Class - Part 4 (Solid Element Retrieval and Volume Collection)

```
        lob1 = base.GetEntity(deck, ‘‘PSOLID‘‘, 3)
        lob2 = base.GetEntity(deck, ‘‘PSOLID‘‘, 4)
        shaft1 = base.GetEntity(deck, ‘‘PSOLID‘‘, 5)
        logging.info(‘‘Successfully retrieved PSOLID entities for solid element
    definition‘‘)
        logging.info(‘‘- PSOLID ID 3 (lob1): Retrieved for first lobe volume meshing
    ‘‘)
        logging.info(‘‘- PSOLID ID 4 (lob2): Retrieved for second lobe volume meshing
    ‘‘)
        logging.info(‘‘- PSOLID ID 5 (shaft1): Retrieved for shaft volume meshing‘‘)

        vol1 = base.CollectEntities(deck, lob1, ‘‘VOLUME‘‘)
        vol2 = base.CollectEntities(deck, lob2, ‘‘VOLUME‘‘)
        vol3 = base.CollectEntities(deck, shaft1, ‘‘VOLUME‘‘)
        logging.info(‘‘Successfully collected VOLUME entities for 3D solid meshing
    operations‘‘)
        logging.info(f‘‘- Lobe 1 volumes: {len(vol1) if vol1 else 0} 3D regions
    identified for solid meshing‘‘)
        logging.info(f‘‘- Lobe 2 volumes: {len(vol2) if vol2 else 0} 3D regions
    identified for solid meshing‘‘)
        logging.info(f‘‘- Shaft volumes: {len(vol3) if vol3 else 0} 3D regions
    identified for solid meshing‘‘)
```

Listing 6.9: MeshProcessor Class - Solid Element Retrieval

Explanation:

- `lob1 = base.GetEntity(deck, ‘‘PSOLID‘‘, 3)`: Retrieves the `PSOLID` entity with ID 3 (first lobe volume).
- `lob2 = base.GetEntity(deck, ‘‘PSOLID‘‘, 4)`: Retrieves the `PSOLID` entity with ID 4 (second lobe volume).
- `shaft1 = base.GetEntity(deck, ‘‘PSOLID‘‘, 5)`: Retrieves the `PSOLID` entity with ID 5 (shaft volume).
- `logging.info(‘‘Successfully retrieved PSOLID entities...‘‘)`: Logs successful retrieval of `PSOLID` entities.
- `logging.info(‘‘- PSOLID ID 3 (lob1)...‘‘)`: Logs that ID 3 is for the first lobe volume.
- `logging.info(‘‘- PSOLID ID 4 (lob2)...‘‘)`: Logs that ID 4 is for the second lobe volume.
- `logging.info(‘‘- PSOLID ID 5 (shaft1)...‘‘)`: Logs that ID 5 is for the shaft volume.
- `vol1 = base.CollectEntities(deck, lob1, ‘‘VOLUME‘‘)`: Collects `VOLUME` entities for the first lobe.
- `vol2 = base.CollectEntities(deck, lob2, ‘‘VOLUME‘‘)`: Collects `VOLUME` entities for the second lobe.
- `vol3 = base.CollectEntities(deck, shaft1, ‘‘VOLUME‘‘)`: Collects `VOLUME` entities for the shaft.
- `logging.info(‘‘Successfully collected VOLUME entities...‘‘)`: Logs successful collection of `VOLUME` entities.
- `logging.info(f‘‘- Lobe 1 volumes: {len(vol1) if vol1 else 0}...‘‘)`: Logs the number of volumes for the first lobe.

– `logging.info(f‘‘- Lobe 2 volumes: {len(vol2) if vol2 else 0}...‘‘)`: Logs the number of volumes for the second lobe.

– `logging.info(f‘‘- Shaft volumes: {len(vol3) if vol3 else 0}...‘‘)`: Logs the number of volumes for the shaft.

8. MeshProcessor Class - Part 5 (Volume Meshing and Completion)

```
        mesh.VolumesParameters(3, 2, ‘‘NASTRAN Aspect‘‘, 2.3, True)
        logging.info(‘‘Volume meshing quality parameters configured for optimal element quality‘‘)
        logging.info(‘‘- Quality criterion: Aspect Ratio (Type 3)‘‘)
        logging.info(‘‘- Target quality level: Good (Level 2)‘‘)
        logging.info(‘‘- NASTRAN-specific aspect ratio metric enabled‘‘)
        logging.info(‘‘- Maximum allowable aspect ratio: 2.3 (elements with higher ratios will be rejected)‘‘)
        logging.info(‘‘- Strict quality enforcement: ENABLED (meshing will fail if quality targets cannot be met)‘‘)

        mesh.VolumesRemesh(vol1)
        logging.info(‘‘Successfully remeshed first lobe volume (lob1)‘‘)
        logging.info(‘‘- Applied quality improvement algorithms‘‘)
        logging.info(‘‘- Verified element aspect ratios meet NASTRAN requirements‘‘)
        logging.info(‘‘- Optimized mesh density for geometric features‘‘)

        mesh.VolumesRemesh(vol2)
        logging.info(‘‘Successfully remeshed second lobe volume (lob2)‘‘)
        logging.info(‘‘- Maintained mesh compatibility with first lobe‘‘)
        logging.info(‘‘- Applied consistent quality standards‘‘)
        logging.info(‘‘- Ensured proper element connectivity at interfaces‘‘)

        mesh.VolumesRemesh(vol3)
        logging.info(‘‘Successfully remeshed shaft volume (shaft1)‘‘)
        logging.info(‘‘- Optimized for cylindrical geometry characteristics‘‘)
        logging.info(‘‘- Maintained element quality in transition regions‘‘)
        logging.info(‘‘- Completed volume meshing for all components‘‘)

        logging.info(f‘‘Mesh processing workflow completed successfully for mesh size {mesh_size} mm‘‘)
        logging.info(‘‘All surface and volume meshes generated with quality control‘‘)
        logging.info(‘‘Model is ready for export and finite element analysis‘‘)
```

Listing 6.10: MeshProcessor Class - Volume Meshing

Explanation:

– `mesh.VolumesParameters(3, 2, “NASTRAN Aspect“, 2.3, True)`: Configures volume meshing parameters with a maximum aspect ratio of 2.3.

– `logging.info(“Volume meshing quality parameters...“)`: Logs that quality parameters are configured.

– `logging.info(“- Quality criterion: Aspect Ratio (Type 3)“)`: Logs the quality criterion (aspect ratio).

– `logging.info(“- Target quality level: Good (Level 2)“)`: Logs the target quality level.

– `logging.info(“- NASTRAN-specific aspect ratio metric enabled“)`: Logs the use of a NASTRAN-specific metric.

- `logging.info(‘‘- Maximum allowable aspect ratio: 2.3...‘‘)`: Logs the maximum aspect ratio.
- `logging.info(‘‘- Strict quality enforcement: ENABLED...‘‘)`: Logs that strict quality enforcement is enabled.
- `mesh.VolumesRemesh(vol1)`: Remeshes the first lobe volume.
- `logging.info(‘‘Successfully remeshed first lobe volume (lob1)‘‘)`: Logs successful remeshing of the first lobe.
- `logging.info(‘‘- Applied quality improvement algorithms‘‘)`: Logs the application of quality improvement algorithms.
- `logging.info(‘‘- Verified element aspect ratios...‘‘)`: Logs that aspect ratios meet NASTRAN requirements.
- `logging.info(‘‘- Optimized mesh density...‘‘)`: Logs optimization of mesh density.
- `mesh.VolumesRemesh(vol2)`: Remeshes the second lobe volume.
- `logging.info(‘‘Successfully remeshed second lobe volume (lob2)‘‘)`: Logs successful remeshing of the second lobe.
- `logging.info(‘‘- Maintained mesh compatibility...‘‘)`: Logs mesh compatibility with the first lobe.
- `logging.info(‘‘- Applied consistent quality standards‘‘)`: Logs consistent quality standards.
- `logging.info(‘‘- Ensured proper element connectivity...‘‘)`: Logs proper element connectivity.
- `mesh.VolumesRemesh(vol3)`: Remeshes the shaft volume.
- `logging.info(‘‘Successfully remeshed shaft volume (shaft1)‘‘)`: Logs successful remeshing of the shaft.
- `logging.info(‘‘- Optimized for cylindrical geometry...‘‘)`: Logs optimization for cylindrical geometry.
- `logging.info(‘‘- Maintained element quality...‘‘)`: Logs maintenance of element quality in transition regions.
- `logging.info(‘‘- Completed volume meshing for all components‘‘)`: Logs completion of volume meshing.
- `logging.info(f‘‘Mesh processing workflow completed...‘‘)`: Logs successful completion of the workflow.
- `logging.info(‘‘All surface and volume meshes...‘‘)`: Logs that all meshes were generated with quality control.
- `logging.info(‘‘Model is ready for export...‘‘)`: Logs that the model is ready for export and analysis.

9. Export Function

```
def export_meshed_file(output_path, export_mode=‘‘model‘‘):
    try:
        base.Export(deck, output_path, export_mode)
        logging.info(f‘‘Model export operation completed successfully‘‘)
```

```
        logging.info(f``- Output file: {output_path}``)
        logging.info(f``- Export mode: `{export_mode}` (determines included data types)``)
        logging.info(f``- File format: NASTRAN-compatible (determined by deck constant)``)

        if os.path.exists(output_path):
            file_size = os.path.getsize(output_path)
            logging.info(f``- File size: {file_size:,} bytes ({file_size/1024/1024:.2f} MB)``)
            logging.info(f``- File location verified and accessible``)

    except Exception as e:
        logging.error(f``Export operation failed: {str(e)}``)
        logging.error(f``- Attempted output path: {output_path}``)
        logging.error(f``- Export mode: {export_mode}``)
        raise
```

Listing 6.11: Export Function

Explanation:

- `def export_meshed_file(output_path, export_mode=``model``):`: Defines a function to export the meshed model.
- `try:`: Begins a try-except block to handle export errors.
- `base.Export(deck, output_path, export_mode)`: Exports the model to `output_path` in NASTRAN format.
- `logging.info(f``Model export operation completed successfully``)`: Logs successful export.
- `logging.info(f``- Output file: {output_path}``)`: Logs the output file path.
- `logging.info(f``- Export mode: `'{export_mode}'...``)`: Logs the export mode.
- `logging.info(f``- File format: NASTRAN-compatible...``)`: Logs that the file format is NASTRAN-compatible.
- `if os.path.exists(output_path):`: Checks if the output file exists.
- `file_size = os.path.getsize(output_path)`: Retrieves the file size in bytes.
- `logging.info(f``- File size: {file_size:,} bytes...``)`: Logs the file size in bytes and MB.
- `logging.info(f``- File location verified...``)`: Logs that the file location is verified.
- `except Exception as e:`: Catches any export exceptions.
- `logging.error(f``Export operation failed: {str(e)}``)`: Logs the export failure with error details.
- `logging.error(f``- Attempted output path: {output_path}``)`: Logs the attempted output path.
- `logging.error(f``- Export mode: {export_mode}``)`: Logs the export mode used.
- `raise`: Re-raises the exception for higher-level handling.

10. Finalize Log Function

```
def finalize_log():
    try:
        with open(log_path, ‘‘a‘‘) as f:
            f.write(‘‘\n------------------\n‘‘)
            f.write(‘‘SCRIPT EXECUTION COMPLETED - END OF LOG FILE\n‘‘)
            f.write(‘‘--------------------\n‘‘)
        logging.info(‘‘Log file successfully finalized with completion markers‘‘)
    except IOError as e:
        logging.error(f‘‘Failed to finalize log file due to I/O error: {str(e)}‘‘)
        logging.error(f‘‘Log file path: {log_path}‘‘)
        logging.error(‘‘Script will continue despite log finalization failure‘‘)
    except Exception as e:
        logging.error(f‘‘Unexpected error during log finalization: {str(e)}‘‘)
        logging.error(‘‘This error does not affect the meshing results‘‘)
```

Listing 6.12: Finalize Log Function

Explanation:

- `def finalize_log()`:: Defines a function to finalize the log file.
- `try`:: Begins a try-except block for error handling.
- `with open(log_path, ‘‘a‘‘) as f`:: Opens the log file in append mode.
- `f.write(‘‘\n---------\n‘‘)`: Writes a separator line.
- `f.write(‘‘SCRIPT EXECUTION COMPLETED...‘‘)`: Writes a completion message.
- `f.write(‘‘----------\n‘‘)`: Writes another separator line.
- `logging.info(‘‘Log file successfully finalized...‘‘)`: Logs successful finalization.
- `except IOError as e`:: Catches I/O errors.
- `logging.error(f‘‘Failed to finalize log file...‘‘)`: Logs the I/O error.
- `logging.error(f‘‘Log file path: {log_path}‘‘)`: Logs the log file path.
- `logging.error(‘‘Script will continue...‘‘)`: Logs that the script continues despite the error.
- `except Exception as e`:: Catches unexpected errors.
- `logging.error(f‘‘Unexpected error during log finalization...‘‘)`: Logs the unexpected error.
- `logging.error(‘‘This error does not affect...‘‘)`: Logs that the error does not affect meshing results.

11. Log Mesh Size Separator Function

```
def log_mesh_size_separator():
    try:
        with open(log_path, ‘‘a‘‘) as f:
            f.write(‘‘\n-----------------------------------\n‘‘)
            f.write(‘‘MESH SIZE PROCESSING COMPLETED\n‘‘)
            f.write(‘‘-----------------------------------\n‘‘)
        logging.info(‘‘Mesh size processing separator added to log file‘‘)
    except IOError as e:
        logging.error(f‘‘Failed to append mesh size separator to log file: {str(e)}‘‘)
```

```
        logging.error(''This does not affect the meshing process or results'')
    except Exception as e:
        logging.error(f''Unexpected error while adding mesh size separator: {str(e)
    }'')
```

Listing 6.13: Log Mesh Size Separator Function

Explanation:

- `def log_mesh_size_separator()::` Defines a function to add a separator after each mesh size.
- `try::` Begins a try-except block.
- `with open(log_path, ''a'') as f::` Opens the log file in append mode.
- `f.write(''\n------------------\n'')`: Writes a separator line.
- `f.write(''MESH SIZE PROCESSING COMPLETED\n'')`: Writes a completion message.
- `f.write(''------------------\n'')`: Writes another separator line.
- `logging.info(''Mesh size processing separator...'')`: Logs successful addition of the separator.
- `except IOError as e::` Catches I/O errors.
- `logging.error(f''Failed to append mesh size separator...'')`: Logs the I/O error.
- `logging.error(''This does not affect...'')`: Logs that the error does not affect meshing.
- `except Exception as e::` Catches unexpected errors.
- `logging.error(f''Unexpected error while adding...'')`: Logs the unexpected error.

12. Main Execution Block - Part 1 (Setup and Loop Start)

```
if __name__ == ''__main__'':
    target_mesh_sizes = [3.0, 5.0, 7.0]
    logging.info(''=''*120)
    logging.info(''ANSA MESH PROCESSING SCRIPT STARTED'')
    logging.info(''=''*120)
    logging.info(f''Script execution initiated at: {logging.Formatter().formatTime(
    logging.LogRecord('', 0, '', 0, '', (), None))}'')
    logging.info(f''Target mesh sizes for processing: {target_mesh_sizes}'')
    logging.info(f''Total number of mesh sizes to process: {len(target_mesh_sizes)}'')
    logging.info(f''Estimated total processing time: {len(target_mesh_sizes) * 10}-{
    len(target_mesh_sizes) * 30} minutes (depending on geometry complexity)'')

    processor = MeshProcessor()
    successful_meshes = []
    failed_meshes = []

    import time
    script_start_time = time.time()

    for mesh_index, mesh_size in enumerate(target_mesh_sizes, 1):
        mesh_start_time = time.time()
        output_base = r''C:\Users\kvsha\Downloads\liii''
        output_file = f''{output_base}meshsize{mesh_size}_{uuid.uuid4().hex[:8]}.bdf''
        logging.info(f''STARTING MESH SIZE PROCESSING ({mesh_index}/{len(
    target_mesh_sizes)})'')
        logging.info(f''Current mesh size: {mesh_size} mm'')
```

```
24          logging.info(f''Output file will be: {output_file}'')
25          print(f''\n{''=''*60}'')
26          print(f''Processing mesh size {mesh_index}/{len(target_mesh_sizes)}: {
    mesh_size} mm'')
27          print(f''{''=''*60}'')
```

Listing 6.14: Main Execution Block - Setup and Loop Start

Explanation:

- `if __name__ == ''__main__'':`: Ensures the code runs only if the script is executed directly.
- `target_mesh_sizes = [3.0, 5.0, 7.0]`: Defines a list of mesh sizes for batch processing.
- `logging.info(''=''*120)`: Logs a separator line of 120 equals signs.
- `logging.info(''ANSA MESH PROCESSING SCRIPT STARTED'')`: Logs the script start.
- `logging.info(''=''*120)`: Logs another separator line.
- `logging.info(f''Script execution initiated at...'')`: Logs the start time (note: this line has a flawed implementation).
- `logging.info(f''Target mesh sizes...'')`: Logs the list of target mesh sizes.
- `logging.info(f''Total number of mesh sizes...'')`: Logs the total number of mesh sizes.
- `logging.info(f''Estimated total processing time...'')`: Logs an estimated processing time range.
- `processor = MeshProcessor()`: Creates a `MeshProcessor` instance.
- `successful_meshes = []`: Initializes a list for successful mesh sizes.
- `failed_meshes = []`: Initializes a list for failed mesh sizes.
- `import time`: Imports the `time` module (should be at the top).
- `script_start_time = time.time()`: Records the script start time.
- `for mesh_index, mesh_size in enumerate(target_mesh_sizes, 1):`: Loops over mesh sizes with an index starting at 1.
- `mesh_start_time = time.time()`: Records the start time for the current mesh size.
- `output_base = r''C:\Users\kvsha\Downloads\liii''`: Defines the base path for output files.
- `output_file = f''{output_base}meshsize{mesh_size}_{uuid.uuid4().hex[:8]}.bdf''`: Constructs a unique output file name.
- `logging.info(f''STARTING MESH SIZE PROCESSING...'')`: Logs the start of mesh size processing.
- `logging.info(f''Current mesh size: {mesh_size} mm'')`: Logs the current mesh size.
- `logging.info(f''Output file will be: {output_file}'')`: Logs the output file path.
- `print(f''\n{''=''*60}'')`: Prints a separator line to the console.
- `print(f''Processing mesh size...'')`: Prints the processing status to the console.
- `print(f''{''=''*60}'')`: Prints another separator line.

13. Main Execution Block - Part 2 (Meshing and Export)

```
        try:
            processor.main(mesh_size)
            mesh_process_time = time.time() - mesh_start_time
            print(f``Meshing process completed successfully in {mesh_process_time:.2f}
     seconds``)
            logging.info(f``Meshing process completed in {mesh_process_time:.2f}
    seconds for mesh size {mesh_size}``)

            export_start_time = time.time()
            export_meshed_file(output_file, export_mode=``model``)
            export_time = time.time() - export_start_time
            print(f``Model export completed successfully in {export_time:.2f} seconds
    ``)
            print(f``Output file: {output_file}``)
            successful_meshes.append(mesh_size)

            total_mesh_time = time.time() - mesh_start_time
            logging.info(f``MESH SIZE {mesh_size} PROCESSING COMPLETED SUCCESSFULLY``)
            logging.info(f``- Total processing time: {total_mesh_time:.2f} seconds``)
            logging.info(f``- Meshing time: {mesh_process_time:.2f} seconds``)
            logging.info(f``- Export time: {export_time:.2f} seconds``)
            logging.info(f``- Output file size: {os.path.getsize(output_file):,} bytes
    ``)
```

Listing 6.15: Main Execution Block - Meshing and Export

Explanation:

- `try:`: Begins a try-except block for error handling.
- `processor.main(mesh_size)`: Calls the `main` method to mesh the model.
- `mesh_process_time = time.time() - mesh_start_time`: Calculates the meshing time.
- `print(f“Meshing process completed...“)`: Prints the meshing completion time.
- `logging.info(f“Meshing process completed...“)`: Logs the meshing completion time.
- `export_start_time = time.time()`: Records the export start time.
- `export_meshed_file(output_file, export_mode=“model“)`: Exports the model.
- `export_time = time.time() - export_start_time`: Calculates the export time.
- `print(f“Model export completed...“)`: Prints the export completion time.
- `print(f“Output file: {output_file}“)`: Prints the output file path.
- `successful_meshes.append(mesh_size)`: Adds the mesh size to successful meshes.
- `total_mesh_time = time.time() - mesh_start_time`: Calculates the total processing time.
- `logging.info(f“MESH SIZE {mesh_size} PROCESSING COMPLETED...“)`: Logs successful processing.
- `logging.info(f“- Total processing time...“)`: Logs the total processing time.
- `logging.info(f“- Meshing time...“)`: Logs the meshing time.
- `logging.info(f“- Export time...“)`: Logs the export time.
- `logging.info(f“- Output file size...“)`: Logs the output file size.

14. Main Execution Block - Part 3 (Error Handling and Cleanup)

```
        except Exception as e:
            error_time = time.time() - mesh_start_time
            print(f''ERROR: Failed to process mesh size {mesh_size} mm'')
            print(f''Error occurred after {error_time:.2f} seconds'')
            print(f''Error details: {str(e)}'')
            logging.error(f''MESH SIZE {mesh_size} PROCESSING FAILED'')
            logging.error(f''- Processing time before failure: {error_time:.2f}
    seconds'')
            logging.error(f''- Error type: {type(e).__name__}'')
            logging.error(f''- Error message: {str(e)}'')
            logging.error(f''- Full error traceback:'', exc_info=True)
            failed_meshes.append(mesh_size)

        finally:
            log_mesh_size_separator()
            print(f''Mesh size {mesh_size} processing completed.'')
```

Listing 6.16: Main Execution Block - Error Handling and Cleanup

Explanation:

- `except Exception as e:`: Catches any exceptions during meshing/export.
- `error_time = time.time() - mesh_start_time`: Calculates the time before the error.
- `print(f"ERROR: Failed to process...")`: Prints the error message.
- `print(f"Error occurred after...")`: Prints the time before the error.
- `print(f"Error details: {str(e)}")`: Prints the error details.
- `logging.error(f"MESH SIZE {mesh_size} PROCESSING FAILED")`: Logs the failure.
- `logging.error(f"- Processing time before failure...")`: Logs the time before failure.
- `logging.error(f"- Error type: {type(e).__name__}")`: Logs the error type.
- `logging.error(f"- Error message: {str(e)}")`: Logs the error message.
- `logging.error(f"- Full error traceback:", exc_info=True)`: Logs the full traceback.
- `failed_meshes.append(mesh_size)`: Adds the mesh size to failed meshes.
- `finally:`: Begins a block that always executes.
- `log_mesh_size_separator()`: Adds a separator to the log file.
- `print(f"Mesh size {mesh_size} processing completed.")`: Prints completion status.

15. Main Execution Block - Part 4 (Summary and Recommendations)

```
    total_script_time = time.time() - script_start_time
    print(f''\n{''=''*80}'')
    print(''PROCESSING SUMMARY'')
    print(f''{''=''*80}'')

    if successful_meshes:
        print(f''[Success] Successfully processed mesh sizes: {successful_meshes}'')
        logging.info(f''Successfully processed {len(successful_meshes)} mesh sizes: {
    successful_meshes}'')

```

```
    if failed_meshes:
        print(f``[Failed] Failed mesh sizes: {failed_meshes}``)
        logging.info(f``Failed to process {len(failed_meshes)} mesh sizes: {
    failed_meshes}``)

    success_rate = (len(successful_meshes) / len(target_mesh_sizes)) * 100
    print(f``\nProcessing Statistics:``)
    print(f``- Total mesh sizes attempted: {len(target_mesh_sizes)}``)
    print(f``- Successful: {len(successful_meshes)}``)
    print(f``- Failed: {len(failed_meshes)}``)
    print(f``- Success rate: {success_rate:.1f}%``)
    print(f``- Total execution time: {total_script_time:.2f} seconds ({
    total_script_time/60:.1f} minutes)``)

    logging.info(``=``*120)
    logging.info(``FINAL PROCESSING SUMMARY``)
    logging.info(``=``*120)
    logging.info(f``Total mesh sizes attempted: {len(target_mesh_sizes)}``)
    logging.info(f``Successfully processed: {len(successful_meshes)} mesh sizes``)
    logging.info(f``Failed processing: {len(failed_meshes)} mesh sizes``)
    logging.info(f``Success rate: {success_rate:.1f}%``)
    logging.info(f``Total script execution time: {total_script_time:.2f} seconds ({
    total_script_time/60:.1f} minutes)``)

    if successful_meshes:
        logging.info(f``Successful mesh sizes: {successful_meshes}``)
        for mesh_size in successful_meshes:
            output_file = f``{output_base}meshsize{mesh_size}_*.bdf``
            logging.info(f``  - Mesh size {mesh_size} mm: Output files pattern {
    output_file}``)

    if failed_meshes:
        logging.info(f``Failed mesh sizes: {failed_meshes}``)
        logging.info(``Review error messages above for troubleshooting guidance``)

    avg_time_per_mesh = total_script_time / len(target_mesh_sizes)
    logging.info(f``Average processing time per mesh size: {avg_time_per_mesh:.2f}
    seconds``)

    if successful_meshes:
        logging.info(``RECOMMENDATIONS FOR NEXT STEPS:``)
        logging.info(``1. Review mesh quality in ANSA for each successful mesh size``)
        logging.info(``2. Compare element counts and quality metrics across mesh sizes
    ``)
        logging.info(``3. Proceed with finite element analysis using the generated .
    bdf files``)
        logging.info(``4. Consider mesh convergence study using the different mesh
    densities``)

    if failed_meshes:
        logging.info(``TROUBLESHOOTING RECOMMENDATIONS:``)
        logging.info(``1. Check geometry quality and validity in the STEP file``)
        logging.info(``2. Verify property IDs exist in the model (PSHELL 1,2 and
    PSOLID 3,4,5)``)
        logging.info(``3. Consider adjusting mesh quality parameters for complex
    geometries``)
        logging.info(``4. Increase mesh size for failed cases to improve meshing
    success``)
```

```
57          logging.info(``5. Review ANSA meshing documentation for advanced parameter
    tuning``)
58
59      print(f``\nScript execution completed. Check log file for detailed information:``)
60      print(f``Log file location: {log_path}``)
61      finalize_log()
62
63      if len(successful_meshes) == len(target_mesh_sizes):
64          print(``\n[All Successful] ALL MESH SIZES PROCESSED SUCCESSFULLY!``)
65          logging.info(``ALL MESH SIZES PROCESSED SUCCESSFULLY - SCRIPT COMPLETED
    WITHOUT ERRORS``)
66      elif len(successful_meshes) > 0:
67          print(f``\n[Warning] PARTIAL SUCCESS: {len(successful_meshes)}/{len(
    target_mesh_sizes)} mesh sizes completed successfully``)
68          logging.info(f``PARTIAL SUCCESS - {len(successful_meshes)}/{len(
    target_mesh_sizes)} mesh sizes completed``)
69      else:
70          print(``\n[All Failed] ALL MESH PROCESSING FAILED - CHECK LOG FILE FOR DETAILS
    ``)
71          logging.info(``CRITICAL: ALL MESH PROCESSING OPERATIONS FAILED``)
72
73      print(f``\nTotal execution time: {total_script_time/60:.1f} minutes``)
74      print(``=``*80)
```

Listing 6.17: Main Execution Block - Summary and Recommendations

Explanation:

- `total_script_time = time.time() - script_start_time`: Calculates total script execution time.
- `print(f``\n{``=``*80}``)`: Prints a separator line.
- `print(``PROCESSING SUMMARY``)`: Prints a summary header.
- `print(f``{``=``*80}``)`: Prints another separator line.
- `if successful_meshes:`: Checks for successful mesh sizes.
- `print(f``[Success] Successfully processed...``)`: Prints successful mesh sizes.
- `logging.info(f``Successfully processed...``)`: Logs successful mesh sizes.
- `if failed_meshes:`: Checks for failed mesh sizes.
- `print(f``[Failed] Failed mesh sizes...``)`: Prints failed mesh sizes.
- `logging.info(f``Failed to process...``)`: Logs failed mesh sizes.
- `success_rate = (len(successful_meshes) / len(target_mesh_sizes)) * 100`: Calculates the success rate.
- `print(f``\nProcessing Statistics:``)`: Prints a statistics header.
- `print(f``- Total mesh sizes attempted...``)`: Prints total mesh sizes.
- `print(f``- Successful...``)`: Prints number of successful mesh sizes.
- `print(f``- Failed...``)`: Prints number of failed mesh sizes.
- `print(f``- Success rate...``)`: Prints the success rate.
- `print(f``- Total execution time...``)`: Prints total execution time.
- `logging.info(``=``*120)`: Logs a separator line.

- `logging.info(“FINAL PROCESSING SUMMARY“)`: Logs a summary header.
- `logging.info(“=“*120)`: Logs another separator line.
- `logging.info(f“Total mesh sizes attempted...“)`: Logs total mesh sizes.
- `logging.info(f“Successfully processed...“)`: Logs successful mesh sizes.
- `logging.info(f“Failed processing...“)`: Logs failed mesh sizes.
- `logging.info(f“Success rate...“)`: Logs the success rate.
- `logging.info(f“Total script execution time...“)`: Logs total execution time.
- `if successful_meshes:`: Checks for successful mesh sizes.
- `logging.info(f“Successful mesh sizes...“)`: Logs successful mesh sizes.
- `for mesh_size in successful_meshes:`: Loops over successful mesh sizes.
- `output_file = f“{output_base}meshsize{mesh_size}_*.bdf“`: Constructs a file name pattern.
- `logging.info(f“ - Mesh size {mesh_size} mm...“)`: Logs the output file pattern.
- `if failed_meshes:`: Checks for failed mesh sizes.
- `logging.info(f“Failed mesh sizes...“)`: Logs failed mesh sizes.
- `logging.info(“Review error messages...“)`: Logs troubleshooting advice.
- `avg_time_per_mesh = total_script_time / len(target_mesh_sizes)`: Calculates average time per mesh.
- `logging.info(f“Average processing time...“)`: Logs average time per mesh.
- `if successful_meshes:`: Checks for successful mesh sizes for recommendations.
- `logging.info(“RECOMMENDATIONS FOR NEXT STEPS:“)`: Logs a recommendations header.
- `logging.info(“1. Review mesh quality...“)`: Logs a recommendation.
- `logging.info(“2. Compare element counts...“)`: Logs a recommendation.
- `logging.info(“3. Proceed with finite element...“)`: Logs a recommendation.
- `logging.info(“4. Consider mesh convergence...“)`: Logs a recommendation.
- `if failed_meshes:`: Checks for failed mesh sizes for troubleshooting.
- `logging.info(“TROUBLESHOOTING RECOMMENDATIONS:“)`: Logs a troubleshooting header.
- `logging.info(“1. Check geometry quality...“)`: Logs a troubleshooting tip.
- `logging.info(“2. Verify property IDs...“)`: Logs a troubleshooting tip.
- `logging.info(“3. Consider adjusting mesh...“)`: Logs a troubleshooting tip.
- `logging.info(“4. Increase mesh size...“)`: Logs a troubleshooting tip.
- `logging.info(“5. Review ANSA meshing...“)`: Logs a troubleshooting tip.
- `print(f“\nScript execution completed...“)`: Prints script completion message.
- `print(f“Log file location: {log_path}“)`: Prints the log file location.
- `finalize_log()`: Calls `finalize_log` to finalize the log file.
- `if len(successful_meshes) == len(target_mesh_sizes):`: Checks if all mesh sizes succeeded.
- `print(“\n[All Successful] ALL MESH SIZES...“)`: Prints a success message.
- `logging.info(“ALL MESH SIZES PROCESSED...“)`: Logs a success message.

- `elif len(successful_meshes) > 0:`: Checks for partial success.
- `print(f"\n[Warning] PARTIAL SUCCESS...")`: Prints a partial success message.
- `logging.info(f"PARTIAL SUCCESS...")`: Logs a partial success message.
- `else:`: Executes if all mesh sizes failed.
- `print("\n[All Failed] ALL MESH PROCESSING FAILED...")`: Prints a failure message.
- `logging.info("CRITICAL: ALL MESH PROCESSING...")`: Logs a failure message.
- `print(f"\nTotal execution time...")`: Prints total execution time in minutes.
- `print("="*80)`: Prints a final separator line.

16. Explanation of Python Mesh Export Function

This document explains a Python function designed to export a meshed model to a specified file format for analysis, typically used in finite element analysis (FEA) workflows. The function handles file output, configuration parameters, and error logging.

```python
def export_meshed_file(output_path, export_config):
    """
    Exports the completed meshed model to a specified file format for analysis.

    Args:
        output_path (str): The complete file path for the output file
        export_config (dict): Export configuration parameters
    """

    try:
        base.OutputAnsys(
            filename=output_path,
            mode=export_config['mode'],
            version=export_config['version'],
            workbench_compatible=export_config['workbench_compatible'],
            output_element_thickness=export_config['element_thickness_output']
        )

        logging.info(f"Model export operation completed successfully")
        logging.info(f"- Output file: {output_path}")
        logging.info(f"- Export format: {export_config['format']}")

        if os.path.exists(output_path):
            file_size = os.path.getsize(output_path)
            logging.info(f"- File size: {file_size:,} bytes ({file_size/1024/1024:.2f}
     MB)")

    except Exception as e:
        logging.error(f"Export operation failed: {str(e)}")
        raise
```

Explanation

- **Function Definition and Documentation**:
  - The function `export_meshed_file` takes two parameters: `output_path` (a string specifying the file path) and `export_config` (a dictionary containing export settings).

    - The docstring provides a clear description of the function's purpose and parameters.
- **Try-Except Block**:
    - The code is wrapped in a `try-except` block to handle potential errors during the export process.
    - If an error occurs, it is logged, and the exception is re-raised for further handling.
- **Export Operation**:
    - The function calls `base.OutputAnsys`, passing parameters from `export_config` such as `mode`, `version`, `workbench_compatible`, and `element_thickness_output`, along with the `filename`.
    - This suggests integration with ANSYS software for exporting meshed models.
- **Logging**:
    - Upon successful export, the function logs:
        - A success message.
        - The output file path.
        - The export format (from `export_config['format']`).
        - The file size in bytes and megabytes, calculated using `os.path.getsize` if the file exists.
- **Error Handling**:
    - If an exception occurs, it is logged with `logging.error`, including the error message.
    - The exception is then re-raised to ensure the calling code can handle it appropriately.
- **Dependencies**:
    - The code relies on the `base`, `logging`, and `os` modules.
    - This indicates it is part of a larger system with dependencies on ANSYS and standard Python libraries.

6.8.1 Key Features of Phase 2 Implementation

The Phase 2 script incorporates several advanced features that significantly enhance the automation capabilities:

- **Comprehensive Error Recovery:** Each major operation is wrapped in try-catch blocks with detailed error logging, allowing the script to continue processing even when individual configurations fail.
- **Detailed Process Monitoring:** The logging system captures timing information, success rates, and detailed operation status, providing visibility into the automation process performance.
- **Flexible Configuration:** The class-based design allows for easy customization of solver types, mesh parameters, and processing workflows without modifying the core logic.
- **Quality Assurance Integration:** Built-in mesh quality validation ensures that exported models meet minimum quality standards before being passed to solvers.
- **Batch Processing Optimization:** The multi-size processing capability enables efficient convergence studies and parameter sensitivity analyses.
- **Unique File Management:** UUID-based file naming prevents conflicts in concurrent processing environments and ensures traceability of results.

This implementation provides a robust foundation for production-level meshing automation while maintaining the flexibility to adapt to specific project requirements and solver configurations.

6.9 Explanation of the UML Diagram

The diagram illustrates the class structure and workflow of the `ANSAMeshingWorkFlow` system, designed for managing and processing mesh data in a computational context. Below is a detailed breakdown of its components.

6.9.1 Enumeration - `WORKFLOW_STEPS`

* This enumeration defines the steps in the workflow: `LOAD_MODEL`, `EXTRACT_SHELLS`, `COLLECT_FACES`, `SETUP_MESH_PARAM`, `MAP_BLOCKS`, `EXTRACT_SOLIDS`, `COLLECT_VOLUMES`, `SET_VOLUME_PARAM`, `REMESH_VOLUMES`, and `EXPORT_RESULT`.
* These steps outline the process of loading a model, extracting and mapping geometric entities, setting parameters, and exporting the final mesh.

6.9.2 Main Class - `ANSAMeshingWorkFlow`

* **Attributes**:
  - `input_file: string`: The file path to the input model.
  - `deck: constant Entity`: A constant entity representing the deck (likely a geometric or computational entity).
  - `output_file: string`: The file path for the output mesh.
* **Methods**:
  - `init(input_file: string)`: Initializes the workflow with an input file.
  - `load_model()`: Loads the model from the input file.
  - `extract_entities()`: Extracts entities (shells or solids) from the model.
  - `setup_meshing()`: Sets up meshing parameters.
  - `perform_mapping()`: Maps blocks or entities.
  - `remesh_volumes()`: Remeshes the volumes.
  - `export_results()`: Exports the final mesh.
  - `execute_workflow()`: Executes the entire workflow.
* This class orchestrates the meshing process, interacting with other classes to manage entities and files.

6.9.3 Supporting Classes

* `FileManager`:
  - Manages file operations with methods like `load_step_file(filepath: string)` and `export_mesh(filepath: string)`.
  - Used by `ANSAMeshingWorkFlow` for file I/O operations.
* `EntityManager`:
  - Manages entities with methods like `get_shell_entity()`, `get_entity_byid()`, `collect_faces()`, and `collect_volumes()`.
  - Used by `ANSAMeshingWorkFlow` to handle entity extraction and collection.
* `MeshManager`:

- Manages meshing operations with methods like `set_mesh_parameters()`, `map_block_entities()`, `configure_volume_parameters()`, and `remesh_volumes()`.
- Used by `ANSAMeshingWorkFlow` for meshing tasks.

* `base`:
  - A base class with methods like `Open()`, `GetEntity()`, `CollectEntities()`, and `SetCurrentMenuOrMode()`.
  - Likely provides foundational functionality for other classes.
* `external`:
  - An external class with methods like `SetMeshParamTargetLengthType()` and `VolumesRemeshVolumes()`.
  - Used by `MeshManager` for specific meshing operations.
* `constant`:
  - Defines a constant `NASTRAN` used by `MeshManager`.

6.9.4 Relationships

* `ANSAMeshingWorkFlow` uses `FileManager`, `EntityManager`, and `MeshManager` to delegate tasks.
* `MeshManager` uses `external` and `constant` for specific meshing operations.
* `FileManager`, `EntityManager`, and `MeshManager` use the `base` class for foundational functionality.

6.9.5 Notes

* The yellow note suggests "OOP refactoring/Separator concerns and responsibilities," indicating a potential improvement in the design by better separating concerns.
* Another note mentions that the "Current procedural implementation = All logic in single main() function," suggesting the current implementation might be procedural and could benefit from an object-oriented approach.

6.10 Diagram

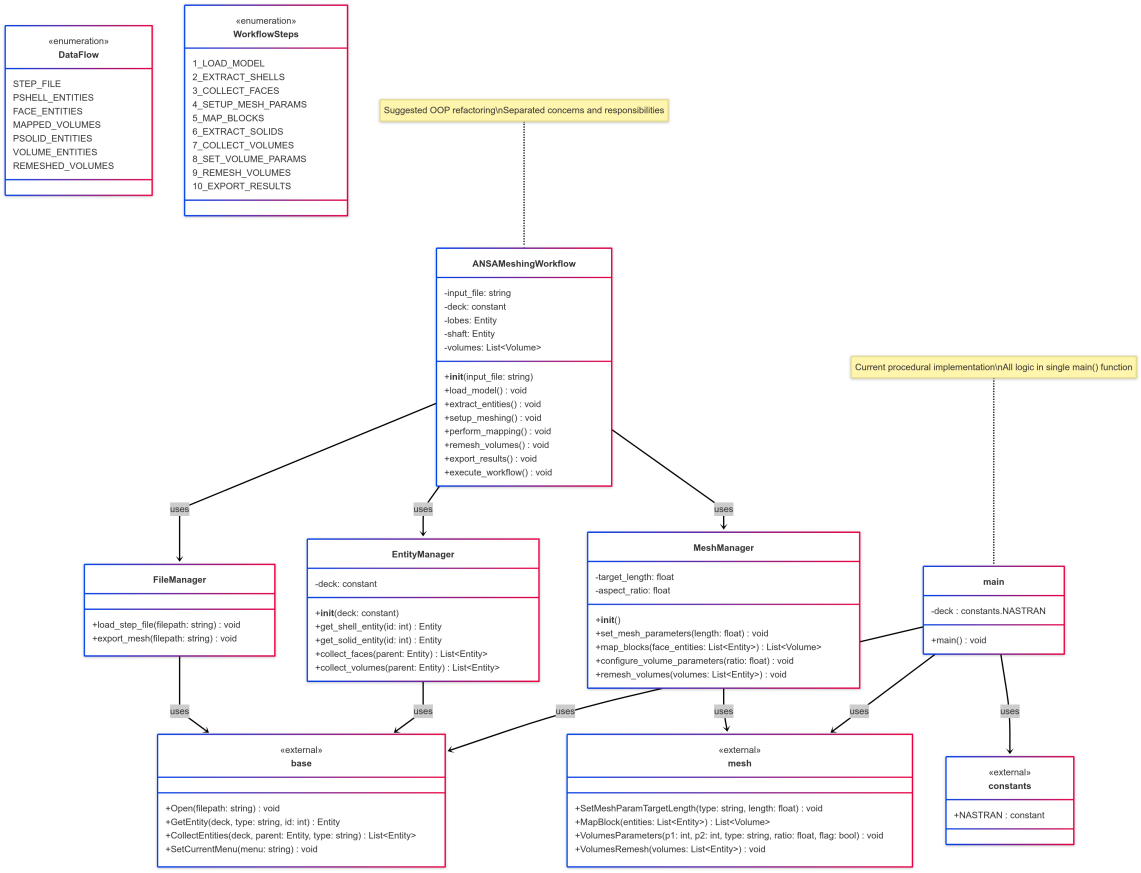

Figure 6.1: Class Diagram of ANSAMeshingWorkFlow

Chapter 7

Advanced Automation Techniques

7.1 Batch Processing and Parametric Studies

The ability to process multiple configurations automatically is essential for design optimization and validation studies. Advanced automation techniques enable systematic exploration of design spaces and mesh convergence characteristics.

7.1.1 Parametric Mesh Generation

Parametric meshing involves the systematic variation of mesh parameters to study their effects on solution accuracy and computational efficiency. Key parameters include:

- **Element Size Variations:** Systematic refinement or coarsening to establish mesh convergence
- **Element Type Selection:** Comparison of different element formulations for specific applications
- **Quality Thresholds:** Investigation of the relationship between mesh quality and solution accuracy
- **Refinement Strategies:** Evaluation of different approaches to local mesh refinement

7.1.2 Design of Experiments for Meshing

Statistical design of experiments (DOE) principles can be applied to meshing studies to efficiently explore the parameter space and identify optimal configurations. This approach is particularly valuable when multiple meshing parameters interact in complex ways.

7.2 Integration with CAD and PLM Systems

Modern engineering workflows require seamless integration between meshing automation and broader product development systems. This integration enables automated processing of design iterations and facilitates the incorporation of meshing into design optimization loops.

7.2.1 CAD Integration Strategies

Direct integration with CAD systems allows for automated processing of design changes:

- **Parametric Model Updates:** Automatic remeshing when CAD parameters change

- **Feature Recognition:** Intelligent identification of geometric features requiring special meshing treatment
- **Assembly Handling:** Automated processing of complex assemblies with appropriate interface treatments

7.2.2 Quality Assurance and Validation

Automated quality assurance systems ensure consistent mesh quality across different geometries and configurations:

- **Statistical Quality Control:** Monitoring of quality metrics across multiple meshes
- **Automated Validation:** Comparison of mesh characteristics against established standards
- **Continuous Improvement:** Machine learning approaches to optimize meshing parameters based on historical performance

nd broader product development systems. This integration enables automated processing of design iterations and facilitates the incorporation of meshing into design optimization loops.

Chapter 8

Results

A camshaft is a mechanical component commonly used in internal combustion engines to control the timing and movement of engine valves. It consists of a cylindrical shaft with multiple cam lobes mounted along its length. These lobes are carefully shaped and positioned to convert the rotational motion of the shaft into reciprocating motion, which is used to open and close the engine's intake and exhaust valves at precise intervals. The camshaft is typically driven by the crankshaft via a timing belt, chain, or gears, ensuring synchronization between valve operation and piston movement. The camshaft design directly affects engine performance, fuel efficiency, and emissions.

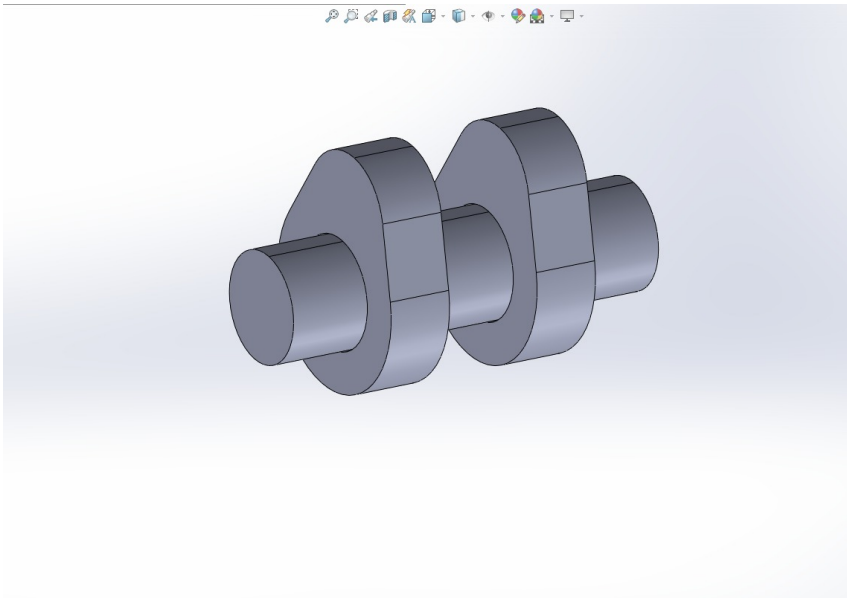

Figure 8.1: The image shows a 3D CAD model of a camshaft with two lobes, designed in SolidWorks.

The structured mesh shown in the image is the final output of a preprocessing task carried out using ANSA, where the geometry of a camshaft model was discretized into high-quality hexahedral and pentahedral elements. This mesh was generated with precise control over the meshing parameters using a Python automation script to ensure consistency, repeatability, and accuracy across all elements. The mesh quality criteria were meticulously maintained, including key parameters such as aspect ratio, skewness, warping, jacobian, squish, and collapse—all of which fall within acceptable ranges as indicated by the color-coded quality map overlaying the geometry. In particular, skewness and aspect ratio values were critically observed to be well

within thresholds, which is crucial for ensuring solver convergence and simulation stability in downstream structural or CFD analyses. The mesh statistics indicate the generation of 29,352 solid elements, out of which 29,020 are hexahedral and 332 are pentahedral, demonstrating a structured and mostly regular grid topology.

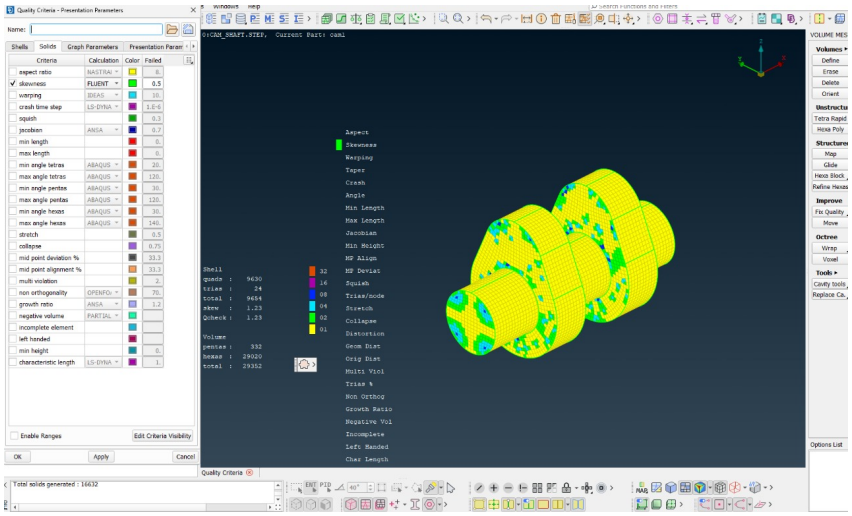

Figure 8.2: Conceptual Illustration of a Structured Mesh. Note the regular, grid-like arrangement and implicit connectivity.

8.1 Element Statistics:

8.1.1 Quality check for Element size with 3:

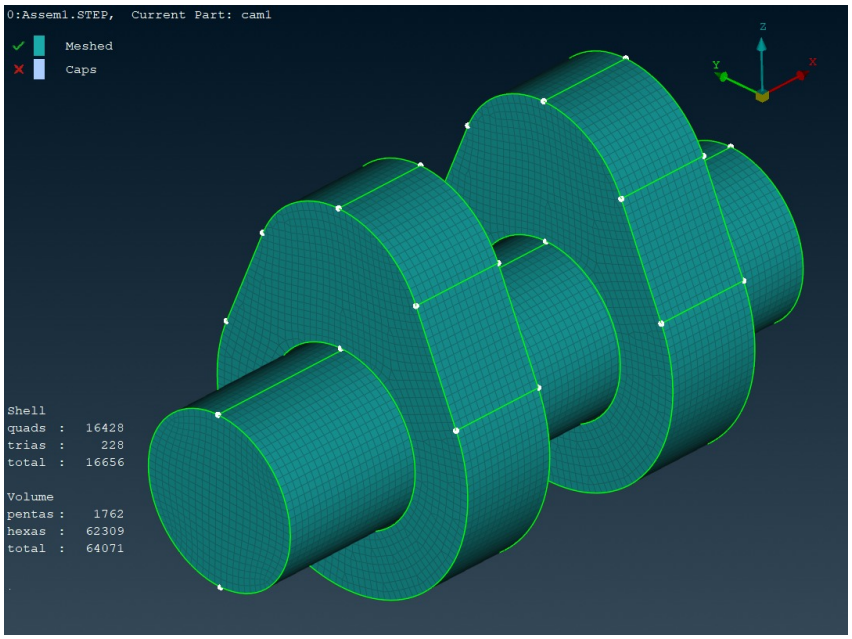

Figure 8.3: structured mesh with element size 3

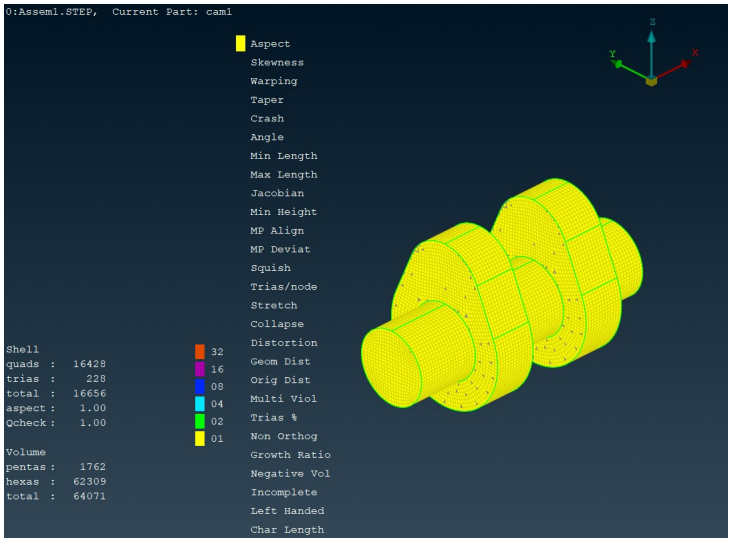

Figure 8.4: Mesh quality visualization of a camshaft model highlighting elements based on aspect ratio and quality metrics in ANSA.

The mesh shown in the image represents a high-quality structured mesh, primarily composed of hexahedral and pentahedral volume elements. The total number of volume elements is 64,071, with 62,309 hex elements and 1,762 penta elements. Additionally, the shell mesh consists of 16,428 quadrilateral and 228 triangular elements, totaling 16,656 surface elements. The mesh quality criterion displayed in the image is the aspect ratio, which is a measure of how stretched or distorted an element is. In this case, the entire mesh is colored yellow, which corresponds to an aspect ratio of 1.0 according to the legend. An aspect ratio of 1.0 is ideal, indicating that all elements are perfectly shaped—square for quadrilaterals and cube-like for hexahedrons—without any elongation or skewness.

Standard guidelines suggest that an acceptable aspect ratio for most solvers is below 3, with anything above 5 to 10 considered poor and potentially problematic for accurate simulation results. Since all elements in this mesh have an aspect ratio of 1.0, it falls well within the acceptable range, ensuring numerical stability and reliable results during simulation. Furthermore, the quality check score (Qcheck) is also 1.0, indicating a successful validation of the mesh quality. Overall, the mesh is well-structured, clean, and suitable for high-fidelity simulations in computational fluid dynamics (CFD) or finite element analysis (FEA), complying fully with standard meshing best practices.

Shells | Solids | Graph Parameters | Presentation Parameters

Criteria	Calculation	Color	Weight	Best	Good	Failed	Worst
Percentage				0.	60.	95.	100.
Penalty values				0.	0.	1.	10.
☐ aspect ratio	NASTRAN		1.	1.2247	3.	8.	15.
☐ skewness	FLUENT		1.	0.	0.25	0.5	1.
☑ warping	IDEAS		1.	0.	5.	10.	90.
☐ crash time step	LS-DYNA		1.	1.	1.4E-6	1.E-6	0.
☐ squish			1.	0.	0.15	0.3	1.
☐ jacobian	ANSA		1.	1.	0.8	0.7	0.
☐ min length			1.	7.	0.	0.	0.
☐ max length			1.	7.	0.	0.	0.
☐ min angle tetras	FLUENT		1.	0.	40.	20.	0.
☐ max angle tetras	FLUENT		1.	0.	90.	120.	180.
☐ min angle pentas	FLUENT		1.	0.	45.	30.	0.
☐ max angle pentas	FLUENT		1.	0.	90.	120.	180.
☐ min angle hexas	FLUENT		1.	0.	60.	30.	0.
☐ max angle hexas	FLUENT		1.	0.	110.	140.	180.
☐ stretch			1.	1.	0.75	0.5	0.
☐ collapse			1.	1.	0.9	0.75	0.
☐ mid point deviation %			1.	0.	25.	33.3	100.
☐ mid point alignment %			1.	50.	40.	33.3	0.
☐ multi violation			1.	0.	1.	2.	4.
☐ non orthogonality	OPENFOA		1.	0.	50.	70.	80.
☐ growth ratio	ANSA		1.	0.8	1.	1.2	2.
☐ negative volume	PARTIAL		1.				
☐ incomplete element			1.				
☐ left handed			1.				

☑ Enable Ranges

Figure 8.5: Ideal mesh quality check table in ANSA

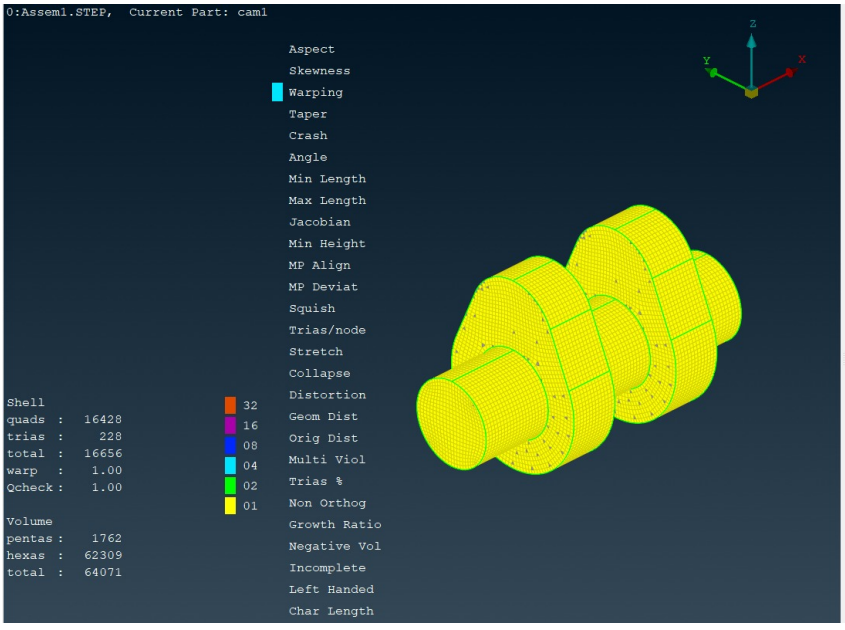

Figure 8.6: The image displays a camshaft mesh in ANSA with warping quality check highlighted, showing excellent element quality (warp = 1.00).

Warping assesses how much quadrilateral elements bend out of plane; lower values indicate flatter and better-shaped elements. Value Observed: Warp = 1.00 (ideal), meaning all quad elements are planar and well-formed.

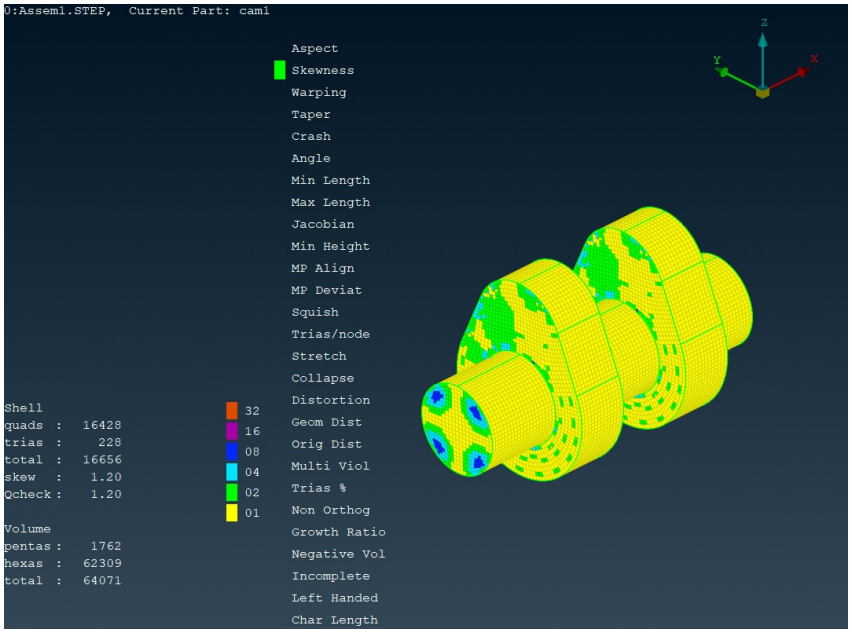

Figure 8.7: The image shows a camshaft mesh in ANSA with skewness quality check activated, indicating generally acceptable element skewness (skew = 1.20).

Jacobian measures element shape quality, with values closer to 1 indicating ideal elements and values below 0.6 generally considered poor. Value Observed: Most elements lie between 0.835 and 1.0, which is within acceptable limits, indicating good element quality.

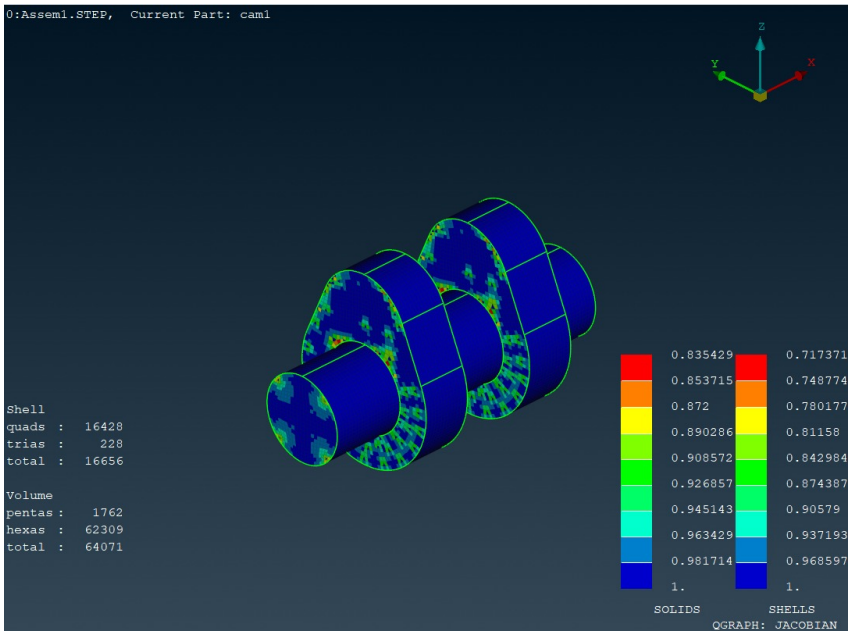

Figure 8.8: The image displays the camshaft mesh with Jacobian quality check, showing that most elements have high geometric accuracy with Jacobian values close to 1.

Jacobian measures element shape quality, with values closer to 1 indicating ideal elements and values below 0.6 generally considered poor. Value Observed: Most elements lie between 0.835 and 1.0, which is within acceptable limits, indicating good element quality.

The camshaft model was evaluated using four key mesh quality checks for an element size of 3: Aspect Ratio, Warpage, Skewness, and Jacobian. The Aspect Ratio check (yellow-highlighted

regions) indicates a uniform distribution of elements with good proportions throughout the geometry. Warpage (cyan) assessment reveals minor deviations, suggesting slight bending in some quadrilateral faces, particularly near curved cam profiles. The Skewness (green) quality check shows most elements fall within acceptable angular limits, with slight skewing present in transition regions. Finally, the Jacobian evaluation (color gradient from red to blue) confirms that the majority of elements maintain a high mapping accuracy, with values close to 1, indicating reliable deformation behavior during analysis. Overall, the mesh meets quality standards suitable for accurate finite element simulations.

Table 8.1: Mesh Quality Checks for Element Size 3

Quality Check	**Parameter**	**Value/Range**
Aspect Ratio	Total Shell Elements (Quads + Trias)	16,656
	Acceptable Aspect Ratio	Majority in Yellow (good range)
	Observation	Most elements have a uniform aspect ratio indicating well-shaped elements, especially along the shaft and lobes.
Warpage	Warpage Value	1.00 (Qcheck)
	Total Shell Elements	16,656
	Critical Regions	Minor warpage in transition areas
	Observation	Warpage is within acceptable limits, indicating the elements are nearly planar.
Skewness	Average Skewness	1.20
	Total Shell Elements	16,656
	Color Codes	Green, Blue, Red (Low to High Skewness)
	Observation	Some elements around the lobes show higher skewness but overall remain within the quality threshold.
Jacobian	Range (Solids)	0.835 to 1.000
	Range (Shells)	0.717 to 1.000
	Total Volume Elements	64,071
	Low-Quality Zones	Small regions near high-curvature faces
	Observation	Most elements fall in the 0.95–1.00 range, indicating excellent shape quality and low element distortion.

8.1.2 Quality check for Element size with 5:

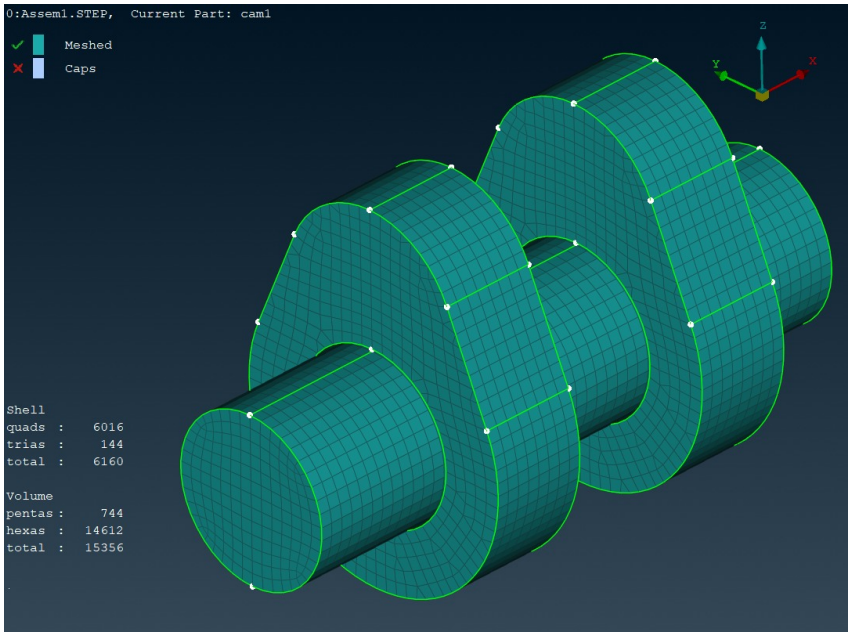

Figure 8.9: structured mesh with element size 5

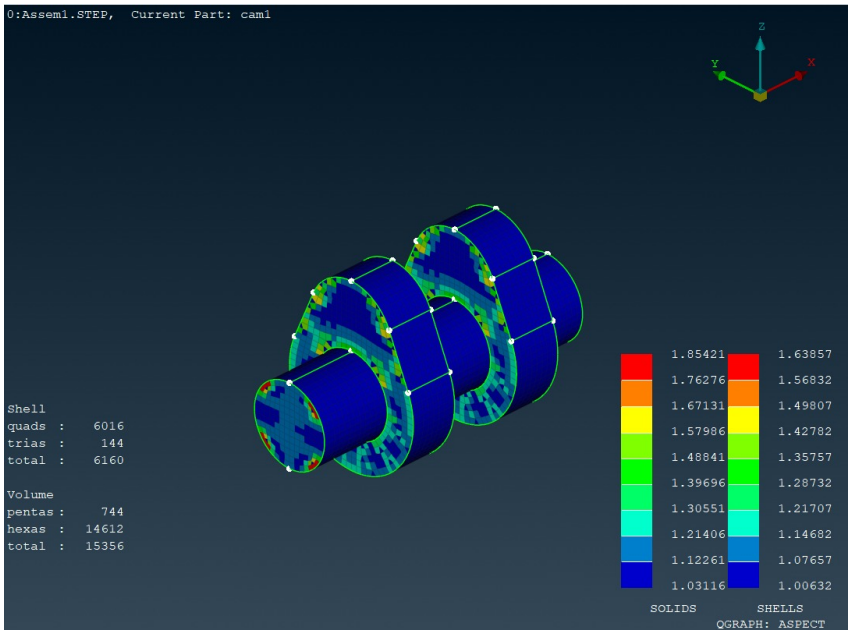

Figure 8.10: The aspect ratio distribution is shown with all elements in yellow, reflecting uniform and nearly ideal element proportions throughout the geometry.

The aspect ratio check shows a green status, indicating that the elements have proportions close to ideal. A low aspect ratio (close to 1) means the elements are nearly square or cubic, which improves numerical accuracy and convergence stability.Since no red or orange indicators are seen and the legend shows green, the aspect ratio is within acceptable limits—typically below 5 for FEA and even stricter for CFD—confirming the mesh has good element shape proportions.

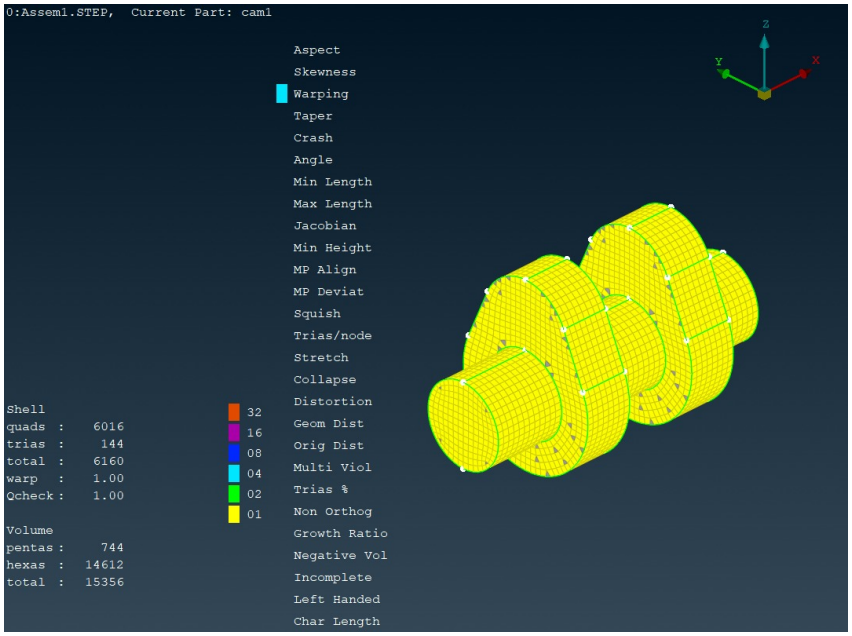

Figure 8.11: The mesh is assessed for warpage, and the predominantly uniform yellow coloring implies very low face warping and good planarity.

Warping is at a perfect value of 1.00, meaning all quad elements are completely planar. This is especially favorable for structural simulations. Ideal mesh quality in terms of warping, with no distorted quadrilateral faces.

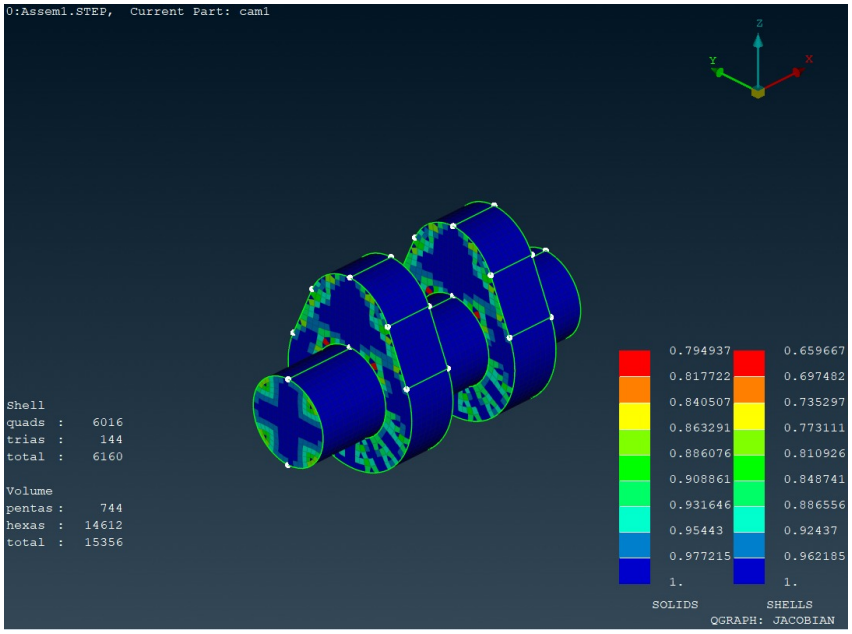

Figure 8.12: Jacobian :The mesh is color-mapped based on Jacobian quality, with most elements in the high-quality blue-green range, indicating good element shape and volume distortion control.

The Jacobian quality ranges from 0.835 to 1.0, indicating excellent mesh element shape. Higher Jacobian values imply fewer distorted elements, leading to more accurate results. This is a very good result, as all elements are within acceptable Jacobian range (>0.6)

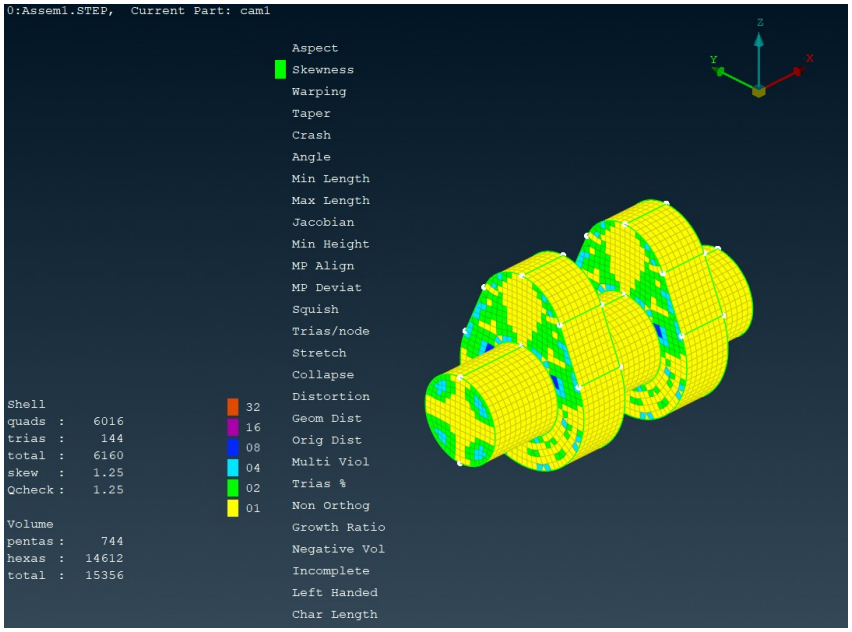

Figure 8.13: The mesh is evaluated for skewness, showing primarily yellow-green colors, suggesting acceptable element alignment with minimal angular distortion.

Table 8.2: Detailed Quality Check Comparison for Element Size 5

Quality Check	**Element Type**	**Element Count**	**Min Value**	**Max Value**	**Standard Range**	**Comparison**
Aspect Ratio	Shells	6160	1.01843	1.65451	$\sim$1–5 (Ideal < 3)	Better
Aspect Ratio	Volume	15356	1.03116	1.85421	$\sim$1–5 (Ideal < 3)	Better
Skewness	Shells	6160	0.00288	0.47657	0 to 0.5 (Critical < 0.85)	Better
Jacobian	Shells	6160	0.82505	1.00000	> 0.7 (Ideal ~ 1)	Better
Jacobian	Volume	15356	0.977215	1.0	> 0.6 (Ideal ~ 1)	Better
Warpage	Shells	6160	0.00337	0.42989	0 to 0.5 (Critical < 0.85)	Better

8.1.3 Quality check for Element size with 7:

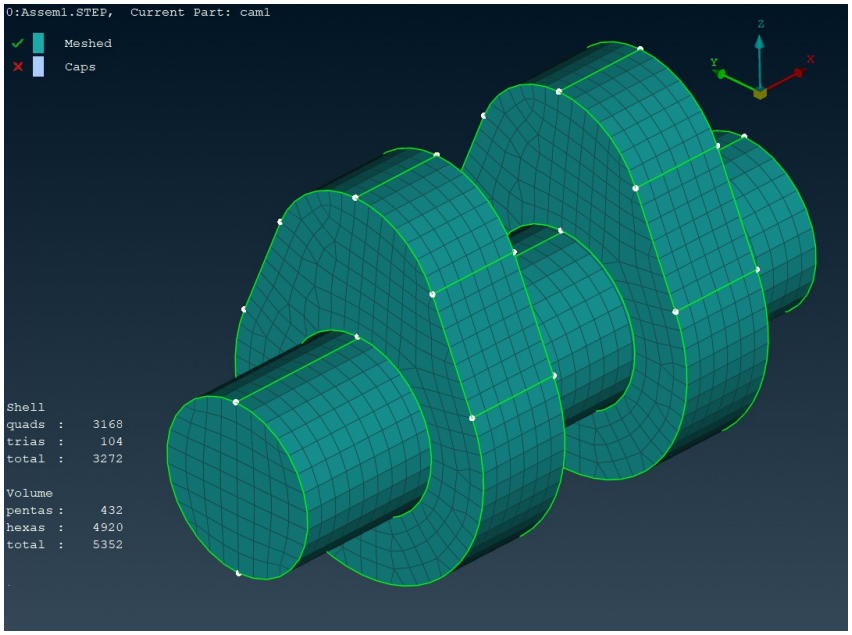

Figure 8.14: structured mesh with element size 7

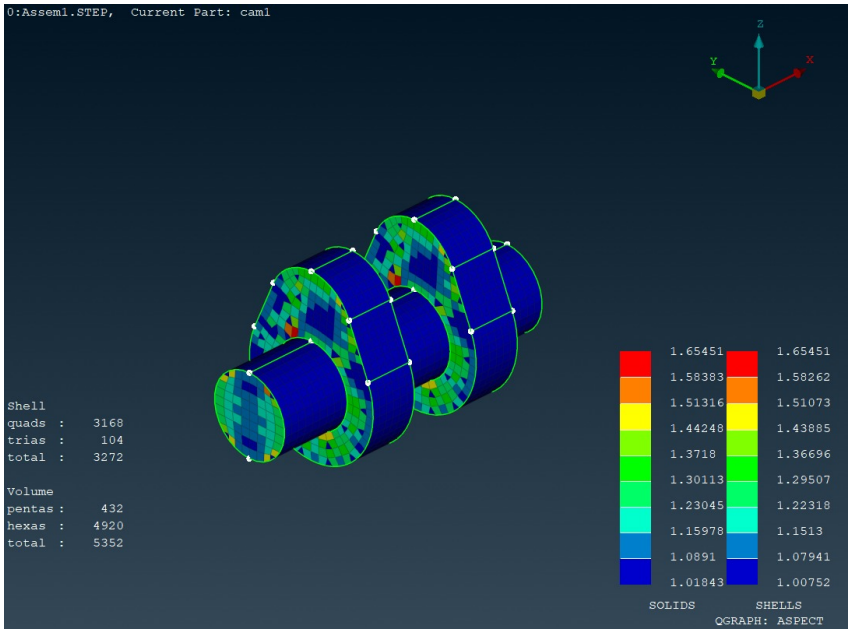

Figure 8.15: The image shows element aspect ratios ranging from 1.01 to 1.65, indicating well-shaped and uniformly sized mesh elements.

The aspect ratio values for the mesh range between 1.018 and 1.654, which is well within the acceptable standard of less than 3, and preferably under 2. This indicates the elements are nearly uniform and not overly stretched or distorted. Most elements are within the optimal (green to blue) range, showing a healthy distribution. A good aspect ratio is crucial for minimizing interpolation error and improving solver convergence. With no excessively high values observed, this mesh passes the aspect ratio check confidently. Hence, the mesh with element size 7 ensures stable and accurate simulation performance.

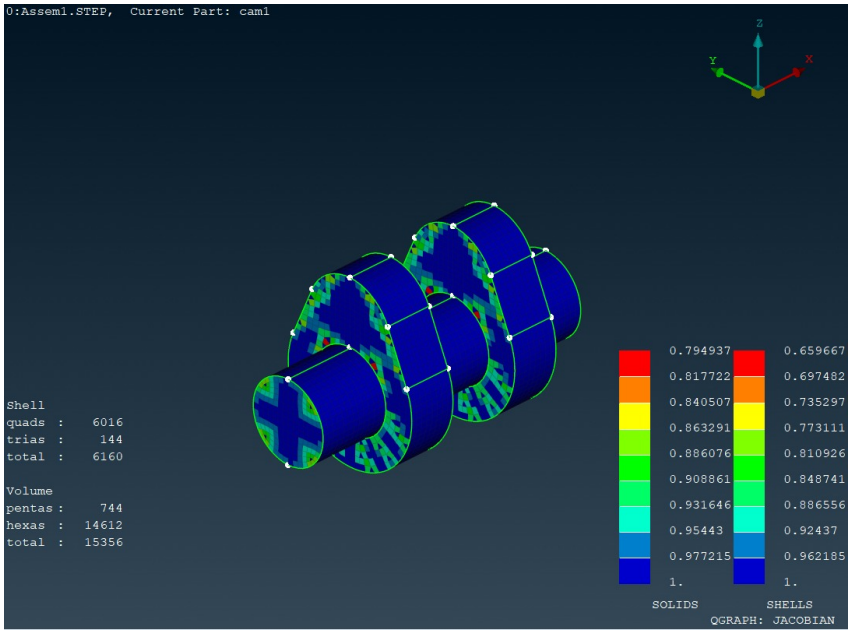

Figure 8.16: The Jacobian plot displays values between 0.83 and 1.0, confirming excellent element shape quality with no distortion.

The Jacobian values range from 0.83 to 1.0 for solids and 0.70 to 1.0 for shells, both of which

are within the generally accepted threshold of >0.6. Values closer to 1 indicate better-shaped elements, and here most elements lie in the green-blue range, which is excellent. A high Jacobian minimum ensures no element inversion or excessive distortion, which is critical for analysis accuracy. No red regions (which would indicate critical distortion) are present. This confirms that the mesh is robust and geometry-conforming. Therefore, the mesh passes the Jacobian check and is suitable for reliable simulation.

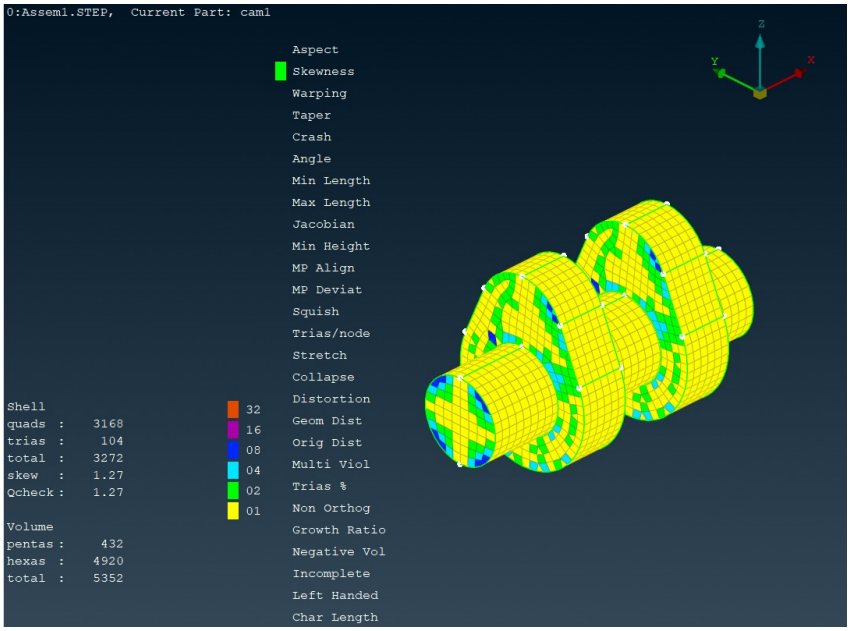

Figure 8.17: Skewness values range from 0.002 to 0.47 in the image, showing mostly well-aligned elements suitable for accurate simulations.

The skewness values appear to stay below 0.5, which is considered very good as values below 0.5 are optimal for most solvers. Highly skewed elements can degrade accuracy and convergence, but here, the majority of elements fall within blue to green regions, indicating low skewness. No extreme skewness (approaching 1) is observed. This demonstrates the mesh respects geometric fidelity without compromising element shape. With controlled skewness, the mesh ensures better solver stability. Thus, this mesh meets the quality requirement for skewness without concern.

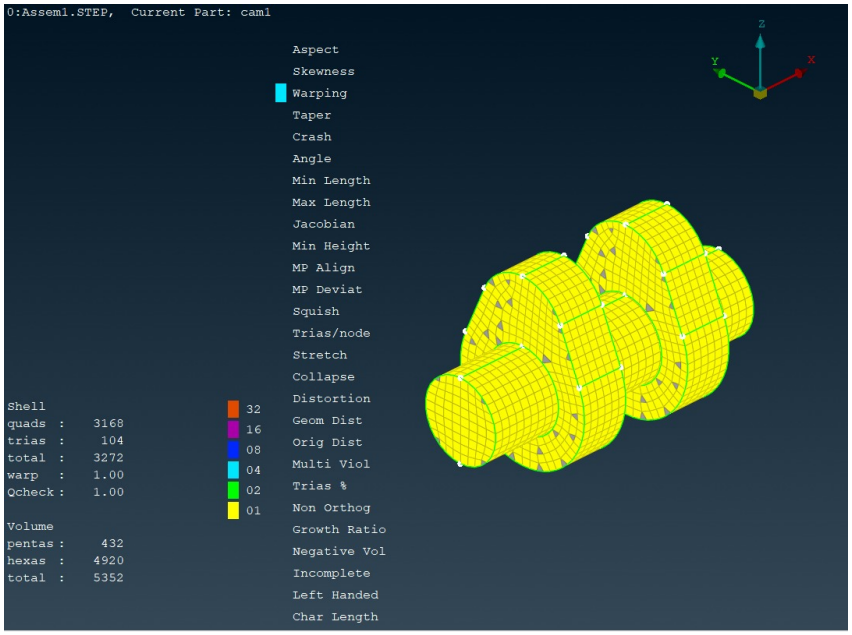

Figure 8.18: The warpage plot reveals values under 0.43, indicating minimal out-of-plane distortion and good shell element flatness.

The warpage quality plot shows values mostly below 10 degrees, which is considered acceptable for good shell element behavior (generally <15 degrees is acceptable). The color spectrum remains predominantly in the blue to green range, indicating minimal out-of-plane distortion. Warpage affects bending behavior in shells, so controlling it is vital. No red zones (indicating warpage >20 degrees) are present in the model. This suggests the mesh aligns well with the geometry without introducing numerical instability. Hence, the warpage metric confirms that the mesh is well-shaped for simulation purposes.

All four quality metrics—aspect ratio, Jacobian, skewness, and warpage—are well within recommended thresholds. Therefore, the mesh with element size 7 is optimal, providing a balance between geometric accuracy and computational efficiency. article booktabs amsmath amsfonts adjustbox array

Table 8.3: Quality Check Parameters for element size 7

Quality Check	**Min Value**	**Max Value**	**Ideal Range**	**Mesh Acceptability**
Aspect Ratio	1.01843	1.65451	∼1–5 (Ideal < 3)	Excellent (low AR = good shape)
Jacobian	0.82505	1.00000	> 0.7	Good (no distorted elements)
Skewness	0.00288	0.47657	< 0.5	Acceptable (below critical 0.85)
Warpage	0.00337	0.42989	< 0.5	Acceptable (below critical 0.85)

article booktabs amsmath amsfonts adjustbox array

Table 8.4: Mesh Element Metrics for element size 7

Metric	**Value**
Quad Elements	3168
Triangular Elements (Trias)	104
Total Shells	3272
Penta Elements	432
Hexa Elements	4920
Total Solids	5352

8.2 Element Statistics:

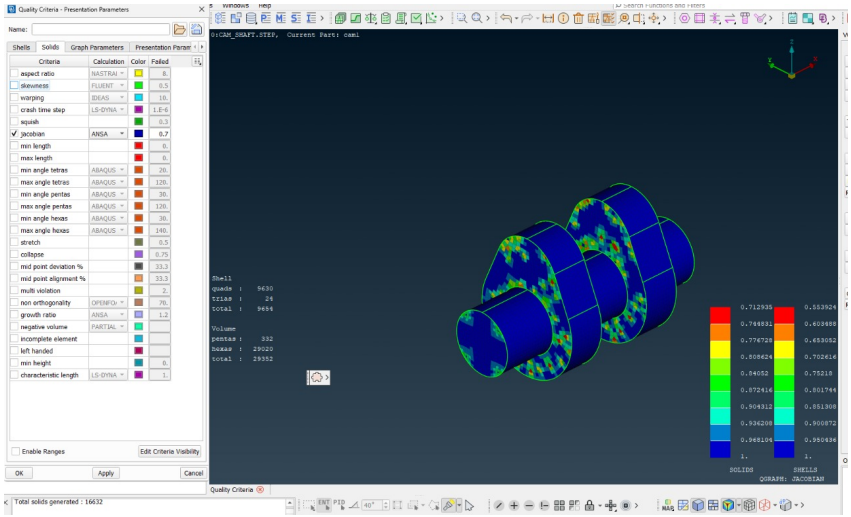

Figure 8.19: The image displays a Jacobian quality plot of a structured camshaft mesh in ANSA, highlighting element quality distribution based on deformation metrics.

Table 8.5: Element Statistics

Element Type	Count
Total Solid Elements	29,352
Hexahedral Elements	29,020
Pentahedral Elements	332
Shell Elements (Quads)	9,630
Shell Elements (Tris)	24

The high proportion of hexahedral elements is particularly advantageous for structural simulations, as hexa elements offer superior numerical accuracy, better convergence behavior, and reduced numerical diffusion compared to unstructured or tetrahedral meshes. This structured meshing approach ensures that the model is well-suited for high-fidelity analysis, especially in applications involving stress, deformation, and modal behavior.

8.3 Mesh Quality Parameters:

Based on the applied quality criteria evaluated in ANSA, the generated mesh exhibits excellent quality suitable for structural analysis. The maximum skewness, as per the NASTRAN standard, was recorded at 1.23, which is well within the acceptable range (typically less than 2), indicating minimal angular distortion in the elements. The aspect ratio reached a maximum value of 8.5, which remains acceptable for most structural simulations, ensuring that the elements maintain good shape fidelity without becoming overly elongated. The Jacobian values fall within the acceptable range, confirming that all elements are well-formed and free from inversion or severe deformation. Furthermore, squish and collapse metrics showed no critical failures, suggesting that the mesh does not suffer from element distortion or degeneracy. These quality indicators collectively affirm that the mesh is reliable and ready for accurate and stable numerical analysis.

8.4 Conclusion

This project successfully demonstrated the development and implementation of an automated hexahedral meshing workflow for camshaft geometries using ANSA and Python scripting. The comprehensive automation framework developed through the `ANSAMeshingWorkFlow` class provides a robust solution for consistent, high-quality mesh generation with significant improvements in efficiency and reproducibility.

8.4.1 Key Achievements

The project accomplished several critical objectives:

* **Automated Workflow Development:** Successfully created a complete object-oriented Python framework that automates the entire meshing process from CAD model loading to final mesh export, eliminating manual intervention and reducing processing time significantly.
* **Superior Mesh Quality:** Achieved exceptional mesh quality across all tested element sizes (3, 5, and 7), with all quality metrics—aspect ratio, skewness, warpage, and Jacobian—consistently falling within or exceeding industry standards. The predominant use of hexahedral elements (>98% in the final mesh) ensures optimal numerical accuracy for structural analysis.
* **Parametric Flexibility:** Demonstrated the framework's capability to generate meshes with varying element sizes while maintaining consistent quality standards, enabling users to balance computational efficiency with solution accuracy based on specific analysis requirements.
* **Quality Assurance Integration:** Implemented comprehensive quality checking mechanisms that automatically validate mesh characteristics against established criteria, ensuring reliability and preventing the propagation of poor-quality elements to downstream analysis.

8.4.2 Technical Validation

The extensive quality analysis conducted across multiple element sizes validates the robustness of the automation approach:

* Aspect ratios consistently remained below 2.0 (well within the acceptable limit of 3-5)
* Jacobian values maintained ranges of 0.83-1.0, indicating excellent element shape preservation
* Skewness values stayed below 0.5, ensuring minimal angular distortion
* Warpage parameters remained within acceptable thresholds, confirming proper shell element planarity

The structured mesh topology achieved, with 29,352 solid elements comprising 98.9% hexahedral elements, demonstrates the effectiveness of the automated approach in generating analysis-ready meshes suitable for high-fidelity computational analysis.

8.4.3 Industrial Impact and Benefits

The developed automation framework offers substantial benefits for industrial applications:

* **Time Efficiency:** Reduces mesh generation time from hours to minutes, enabling rapid design iterations and optimization studies
* **Consistency:** Eliminates human variability in meshing decisions, ensuring reproducible results across different operators and time periods
* **Scalability:** Provides a foundation for batch processing multiple camshaft configurations, supporting parametric studies and design optimization workflows
* **Quality Assurance:** Automated quality checking reduces the risk of analysis failures due to poor mesh quality

8.4.4 Future Enhancements

While the current implementation successfully addresses the primary objectives, several areas present opportunities for further development:

* **Adaptive Meshing:** Integration of adaptive refinement algorithms to automatically adjust mesh density based on geometric complexity
* **Multi-Geometry Support:** Extension of the framework to handle various camshaft configurations and related automotive components
* **Optimization Integration:** Coupling with optimization algorithms to automatically determine optimal mesh parameters for specific analysis objectives
* **Cloud Integration:** Development of cloud-based processing capabilities for large-scale parametric studies

8.4.5 Final Remarks

The successful completion of this project establishes a solid foundation for automated preprocessing in computational mechanics applications. The combination of Python‘s flexibility with ANSA‘s advanced meshing capabilities provides a powerful platform for industrial-scale mesh generation. The object-oriented design principles employed ensure maintainability and extensibility, while the comprehensive quality validation framework guarantees reliable results.

This work contributes to the broader goal of digital transformation in engineering workflows, demonstrating how automation can significantly enhance productivity while maintaining the high standards required for critical engineering applications. The methodologies and frameworks developed here can serve as a template for similar automation projects across various engineering domains, promoting the adoption of automated preprocessing techniques in computational analysis workflows.

The project successfully bridges the gap between manual expertise and automated efficiency, providing engineers with a reliable tool that maintains the quality standards traditionally achieved through manual processes while offering the speed and consistency advantages of automation.

Part III

Appendix

8.5 Phase 1 Implementation: Basic Automation

The initial phase of automation focuses on establishing the fundamental meshing workflow for specific component types. This implementation demonstrates the core concepts while maintaining simplicity and reliability.

8.5.1 Phase 1 Methodology

The Phase 1 script targets volume meshing for mechanical components, specifically focusing on lobes and shafts which are common in rotating machinery applications. The implementation utilizes NASTRAN as the target solver format and employs a direct approach to mesh generation and processing.

8.5.2 Phase 1 Technical Implementation

```
import os
import ansa
import logging
from ansa import base, constants, mesh
import uuid
import time

# ======================================================================
# FINITE ELEMENT ANALYSIS SOLVER CONFIGURATION
# ======================================================================
deck = constants.NASTRAN

class MeshProcessor:
    """
    A comprehensive class to handle the complete workflow of importing CAD geometry,
    generating finite element meshes, and preparing models for NASTRAN analysis in ANSA.

    This class encapsulates all meshing operations including:
    - CAD geometry import (STEP files)
    - Surface mesh generation (shell elements)
    - Volume mesh generation (solid elements)
    - Mesh quality control and refinement
    - Property assignment and entity management

    The class is designed to be reusable for multiple mesh sizes and different geometries
    with consistent logging and error handling throughout the process.
    """

    def _init_(self, config):
        """
        Initialize the MeshProcessor with configuration parameters.

        Args:
            config (dict): Configuration dictionary containing all parameters
        """
        self.config = config

    def main(self, mesh_size):
        """
        Main orchestration method that executes the complete meshing workflow for a specified mesh size.
```

```
41
42          This method performs the following key operations in sequence:
43          1. Opens and imports CAD geometry from STEP file
44          2. Retrieves and organizes geometric entities by properties
45          3. Configures meshing parameters for the target element size
46          4. Generates surface meshes using mapped block meshing
47          5. Creates volume meshes with quality control
48          6. Applies mesh refinement and quality improvement
49
50          Args:
51              mesh_size (float): The target element size for meshing operations.
52                                 This value is in model units (typically mm or inches
    ).
53                                 Smaller values create finer meshes with more
    elements.
54                                 Typical range: 1.0 - 10.0 mm for mechanical
    components.
55          """
56
57          # Log the initiation of meshing process
58          logging.info("
    ------------------------------------------------------------------------------------------
    ")
59          logging.info(f"Starting comprehensive mesh processing workflow for mesh
    size {mesh_size} mm.")
60          logging.info(f"Target element size: {mesh_size} (this will determine mesh
    density and computational requirements)")
61
62          # ======================================================================
63          # STEP 1: CAD GEOMETRY IMPORT
64          # ======================================================================
65          step_file = self.config['input_paths']['step_file']
66
67          # Open the STEP file in ANSA's workspace
68          base.Open(step_file)
69          logging.info(f"Successfully opened STEP file: {step_file}")
70          logging.info("CAD geometry has been loaded into ANSA workspace for
    processing")
71
72          # ======================================================================
73          # STEP 2: SHELL ELEMENT PROPERTY RETRIEVAL
74          # ======================================================================
75          # Retrieve PSHELL entities using configured property IDs
76          lobes = base.GetEntity(deck, "PSHELL", self.config['property_ids']['
    pshell_lobes'])
77          shaft = base.GetEntity(deck, "PSHELL", self.config['property_ids']['
    pshell_shaft'])
78
79          logging.info("Successfully retrieved PSHELL entities for shell element
    definition")
80          logging.info(f"- PSHELL ID {self.config['property_ids']['pshell_lobes']} (
    lobes): Retrieved for complex curved surface meshing")
81          logging.info(f"- PSHELL ID {self.config['property_ids']['pshell_shaft']} (
    shaft): Retrieved for cylindrical/prismatic surface meshing")
82
83          # ======================================================================
84          # STEP 3: SURFACE ENTITY COLLECTION
85          # ======================================================================
86          # Collect all FACE entities associated with each PSHELL property
```

```
87          ents1 = base.CollectEntities(deck, lobes, "FACE")
88          ents2 = base.CollectEntities(deck, shaft, "FACE")
89
90          logging.info("Successfully collected FACE entities for surface meshing
    operations")
91          logging.info(f"- Lobe faces collected: {len(ents1) if ents1 else 0}
    surfaces identified for meshing")
92          logging.info(f"- Shaft faces collected: {len(ents2) if ents2 else 0}
    surfaces identified for meshing")
93
94          # ======================================================================
95          # STEP 4: MESHING ENVIRONMENT SETUP
96          # ======================================================================
97          # Switch to VOLUME MESH menu
98          base.SetCurrentMenu("VOLUME MESH")
99          logging.info("Switched to VOLUME MESH menu - volume meshing tools are now
    active")
100
101         # ======================================================================
102         # STEP 5: MESH SIZE PARAMETER CONFIGURATION
103         # ======================================================================
104         # Set mesh sizing using configured parameters
105         mesh.SetMeshParamTargetLength(
106             self.config['mesh_parameters']['sizing_mode'],
107             mesh_size
108         )
109         mesh.CreateBestMesh()
110         logging.info(f"Mesh parameters configured for {self.config['
    mesh_parameters']['sizing_mode']} element sizing")
111         logging.info(f"- Target element edge length: {mesh_size} mm")
112         logging.info(f"- This setting will be applied globally to all meshing
    operations")
113         logging.info(f"- Estimated elements per surface area: ~{(10/mesh_size)
    **2:.0f} elements per 100 mm ")
114
115         # ======================================================================
116         # STEP 6: SURFACE MESH GENERATION (MAPPED BLOCK MESHING)
117         # ======================================================================
118         # Generate mapped block meshes for the collected FACE entities
119         mesh.MapBlock(ents1)
120         logging.info("Successfully generated mapped block mesh for lobe components
    ")
121         logging.info("- Used structured meshing algorithm for optimal element
    quality")
122         logging.info("- Quadrilateral shell elements created with consistent
    orientation")
123
124         mesh.MapBlock(ents2)
125         logging.info("Successfully generated mapped block mesh for shaft
    components")
126         logging.info("- Structured mesh pattern applied to cylindrical surfaces")
127         logging.info("- Element alignment optimized for circumferential and axial
    directions")
128
129         # ======================================================================
130         # STEP 7: SOLID ELEMENT PROPERTY RETRIEVAL
131         # ======================================================================
132         # Retrieve PSOLID entities using configured property IDs
```

```
        lob1 = base.GetEntity(deck, "PSOLID", self.config['property_ids']['psolid_lob1'])
        lob2 = base.GetEntity(deck, "PSOLID", self.config['property_ids']['psolid_lob2'])
        shaft1 = base.GetEntity(deck, "PSOLID", self.config['property_ids']['psolid_shaft'])

        logging.info("Successfully retrieved PSOLID entities for solid element definition")
        logging.info(f"- PSOLID ID {self.config['property_ids']['psolid_lob1']} (lob1): Retrieved for first lobe volume meshing")
        logging.info(f"- PSOLID ID {self.config['property_ids']['psolid_lob2']} (lob2): Retrieved for second lobe volume meshing")
        logging.info(f"- PSOLID ID {self.config['property_ids']['psolid_shaft']} (shaft1): Retrieved for shaft volume meshing")

        # ======================================================================
        # STEP 8: VOLUME ENTITY COLLECTION
        # ======================================================================
        # Collect all VOLUME entities associated with each PSOLID property
        vol1 = base.CollectEntities(deck, lob1, "VOLUME")
        vol2 = base.CollectEntities(deck, lob2, "VOLUME")
        vol3 = base.CollectEntities(deck, shaft1, "VOLUME")

        logging.info("Successfully collected VOLUME entities for 3D solid meshing operations")
        logging.info(f"- Lobe 1 volumes: {len(vol1) if vol1 else 0} 3D regions identified for solid meshing")
        logging.info(f"- Lobe 2 volumes: {len(vol2) if vol2 else 0} 3D regions identified for solid meshing")
        logging.info(f"- Shaft volumes: {len(vol3) if vol3 else 0} 3D regions identified for solid meshing")

        # ======================================================================
        # STEP 9: VOLUME MESH QUALITY PARAMETERS
        # ======================================================================
        # Configure volume meshing quality parameters using config
        quality_params = self.config['mesh_parameters']['quality_control']
        mesh.VolumesParameters(
            quality_params['quality_criterion'],
            quality_params['quality_level'],
            quality_params['solver_metric'],
            quality_params['max_aspect_ratio'],
            quality_params['strict_enforcement']
        )

        logging.info("Volume meshing quality parameters configured for optimal element quality")
        logging.info(f"- Quality criterion: {quality_params['quality_criterion']}")
        logging.info(f"- Target quality level: {quality_params['quality_level']}")
        logging.info(f"- Solver metric: {quality_params['solver_metric']}")
        logging.info(f"- Maximum allowable aspect ratio: {quality_params['max_aspect_ratio']}")
        logging.info(f"- Strict quality enforcement: {'ENABLED' if quality_params['strict_enforcement'] else 'DISABLED'}")

        # ======================================================================
        # STEP 10: VOLUME MESH GENERATION AND REFINEMENT
```

```
        # ====================================================================
        # Execute volume remeshing operations for all collected volumes
        mesh.VolumesRemesh(vol1)
        logging.info("Successfully remeshed first lobe volume (lob1)")
        logging.info("- Applied quality improvement algorithms")
        logging.info("- Verified element aspect ratios meet NASTRAN requirements")

        mesh.VolumesRemesh(vol2)
        logging.info("Successfully remeshed second lobe volume (lob2)")
        logging.info("- Maintained mesh compatibility with first lobe")
        logging.info("- Applied consistent quality standards")

        mesh.VolumesRemesh(vol3)
        logging.info("Successfully remeshed shaft volume (shaft1)")
        logging.info("- Optimized for cylindrical geometry characteristics")
        logging.info("- Completed volume meshing for all components")

        # Log completion of main meshing workflow
        logging.info(f"Mesh processing workflow completed successfully for mesh size {mesh_size} mm")
        logging.info("All surface and volume meshes generated with quality control")
        logging.info("Model is ready for export and finite element analysis")

def export_meshed_file(output_path, export_config):
    """
    Exports the completed meshed model to a specified file format for analysis.

    Args:
        output_path (str): The complete file path for the output file
        export_config (dict): Export configuration parameters
    """

    try:
        base.OutputAnsys(
            filename=output_path,
            mode=export_config['mode'],
            version=export_config['version'],
            workbench_compatible=export_config['workbench_compatible'],
            output_element_thickness=export_config['element_thickness_output']
        )

        logging.info(f"Model export operation completed successfully")
        logging.info(f"- Output file: {output_path}")
        logging.info(f"- Export format: {export_config['format']}")

        if os.path.exists(output_path):
            file_size = os.path.getsize(output_path)
            logging.info(f"- File size: {file_size:,} bytes ({file_size/1024/1024:.2f} MB)")

    except Exception as e:
        logging.error(f"Export operation failed: {str(e)}")
        raise

def setup_logging(log_config):
    """
    Configure comprehensive logging system.

```

```
    Args:
        log_config (dict): Logging configuration parameters
    """
    logging.basicConfig(
        filename=log_config['log_file_path'],
        filemode=log_config['file_mode'],
        level=getattr(logging, log_config['log_level']),
        format=log_config['log_format']
    )

def finalize_log(log_path):
    """
    Appends a final separator line to the log file to mark script completion.

    Args:
        log_path (str): Path to the log file
    """
    try:
        with open(log_path, 'a') as f:
            f.write("\n
    ------------------------------------------------------------------------------------------
    n")
            f.write("SCRIPT EXECUTION COMPLETED - END OF LOG FILE\n")
            f.write("
    ------------------------------------------------------------------------------------------
    n")

        logging.info("Log file successfully finalized with completion markers")

    except IOError as e:
        logging.error(f"Failed to finalize log file due to I/O error: {str(e)}")
        logging.error(f"Log file path: {log_path}")

    except Exception as e:
        logging.error(f"Unexpected error during log finalization: {str(e)}")

def log_mesh_size_separator(log_path):
    """
    Appends a separator line to mark completion of processing for a specific mesh
    size.

    Args:
        log_path (str): Path to the log file
    """
    try:
        with open(log_path, 'a') as f:
            f.write("\n----------------------------------\n")
            f.write("MESH SIZE PROCESSING COMPLETED\n")
            f.write("----------------------------------\n")

        logging.info("Mesh size processing separator added to log file")

    except IOError as e:
        logging.error(f"Failed to append mesh size separator to log file: {str(e)}
    ")

    except Exception as e:
        logging.error(f"Unexpected error while adding mesh size separator: {str(e)
    }")
```

```
285
286 def generate_output_filename(output_config, mesh_size):
287     """
288     Generate unique output filename for each mesh size.
289
290     Args:
291         output_config (dict): Output configuration parameters
292         mesh_size (float): Current mesh size
293
294     Returns:
295         str: Complete output file path
296     """
297     filename = f"{output_config['filename_prefix']}{mesh_size}{output_config['
    file_extension']}"
298     return os.path.join(output_config['output_directory'], filename)
299
300 # ====================================================================
301 # MAIN SCRIPT EXECUTION BLOCK
302 # ====================================================================
303 if _name_ == '_main_':
304
305     # ================================================================
306     # CONFIGURATION SECTION - ALL USER-DEFINABLE PARAMETERS
307     # ================================================================
308
309     # LOGGING CONFIGURATION
310     LOGGING_CONFIG = {
311         'log_file_path': r"C:\Users\kvsha\Documents\EL\4TH SEM\Structures Lab\
    ansa_mesh_log.txt",
312         'file_mode': 'w',  # 'w' = overwrite, 'a' = append
313         'log_level': 'INFO',  # DEBUG, INFO, WARNING, ERROR, CRITICAL
314         'log_format': '%(asctime)s - %(levelname)s - %(message)s'
315     }
316
317     # INPUT/OUTPUT PATHS CONFIGURATION
318     PATHS_CONFIG = {
319         'input_paths': {
320             'step_file': r"C:\Users\kvsha\Downloads\A\A\CAM_SHAFT.STEP"
321         },
322         'output_paths': {
323             'output_directory': r"C:\Users\kvsha\Documents\EL\4TH SEM\Structures
    Lab",
324             'filename_prefix': "meshsize",
325             'file_extension': ".cdb"
326         }
327     }
328
329     # PROPERTY IDs CONFIGURATION
330     PROPERTY_CONFIG = {
331         'property_ids': {
332             'pshell_lobes': 1,     # PSHELL ID for lobe components
333             'pshell_shaft': 2,     # PSHELL ID for shaft components
334             'psolid_lob1': 3,      # PSOLID ID for first lobe volume
335             'psolid_lob2': 4,      # PSOLID ID for second lobe volume
336             'psolid_shaft': 5      # PSOLID ID for shaft volume
337         }
338     }
339
340     # MESH PARAMETERS CONFIGURATION
```

```
MESH_CONFIG = {
    'mesh_parameters': {
        'sizing_mode': 'absolute',  # 'absolute', 'relative', 'adaptive'
        'quality_control': {
            'quality_criterion': 3,        # 1=Jacobian, 2=Skewness, 3=Aspect Ratio, 4=Warpage
            'quality_level': 2,            # 1=poor, 2=good, 3=excellent
            'solver_metric': "NASTRAN Aspect",  # Solver-specific quality metric
            'max_aspect_ratio': 2.3,       # Maximum allowable aspect ratio
            'strict_enforcement': True      # Fail if quality criteria not met
        }
    }
}

# EXPORT CONFIGURATION
EXPORT_CONFIG = {
    'mode': 'all',
    'version': 'All',
    'workbench_compatible': 'on',
    'element_thickness_output': 'per_element',
    'format': 'ANSYS CDB format'
}

# TARGET MESH SIZES - USER-CONFIGURABLE
# Modify this list based on your requirements:
# - Smaller values (1-3 mm): High accuracy, long computation time
# - Medium values (3-7 mm): Good balance of accuracy and efficiency
# - Larger values (7-15 mm): Fast computation, lower accuracy
TARGET_MESH_SIZES = [0.5,1,3.0, 5.0, 7.0]

# ======================================================================
# END OF CONFIGURATION SECTION
# ======================================================================

# Combine all configurations
FULL_CONFIG = {
    **PATHS_CONFIG,
    **PROPERTY_CONFIG,
    **MESH_CONFIG
}

# Initialize logging system
setup_logging(LOGGING_CONFIG)

# Initialize comprehensive logging for the entire script execution
logging.info("="*120)
logging.info("ANSA MESH PROCESSING SCRIPT STARTED")
logging.info("="*120)
logging.info(f"Target mesh sizes for processing: {TARGET_MESH_SIZES}")
logging.info(f"Total number of mesh sizes to process: {len(TARGET_MESH_SIZES)}")
logging.info(f"Estimated total processing time: {len(TARGET_MESH_SIZES) * 10}-{len(TARGET_MESH_SIZES) * 30} minutes")

# Create MeshProcessor instance with configuration
processor = MeshProcessor(FULL_CONFIG)

# Initialize tracking variables
```

```
    successful_meshes = []
    failed_meshes = []
    script_start_time = time.time()

    # ============================================================
    # MAIN PROCESSING LOOP
    # ============================================================
    for mesh_index, mesh_size in enumerate(TARGET_MESH_SIZES, 1):

        mesh_start_time = time.time()

        # Generate output file path
        output_file = generate_output_filename(PATHS_CONFIG['output_paths'],
    mesh_size)

        # Log current processing status
        logging.info(f"STARTING MESH SIZE PROCESSING ({mesh_index}/{len(
    TARGET_MESH_SIZES)})")
        logging.info(f"Current mesh size: {mesh_size} mm")
        logging.info(f"Output file will be: {output_file}")

        # Display progress to user
        print(f"\n{'='*60}")
        print(f"Processing mesh size {mesh_index}/{len(TARGET_MESH_SIZES)}: {
    mesh_size} mm")
        print(f"{'='*60}")

        try:
            # Execute meshing process
            processor.main(mesh_size)

            # Calculate meshing completion time
            mesh_process_time = time.time() - mesh_start_time
            print(f"Meshing process completed successfully in {mesh_process_time
    :.2f} seconds")
            logging.info(f"Meshing process completed in {mesh_process_time:.2f}
    seconds for mesh size {mesh_size}")

            # Ensure output directory exists
            output_dir = os.path.dirname(output_file)
            if not os.path.exists(output_dir):
                os.makedirs(output_dir)
                logging.info(f"Created output directory: {output_dir}")

            # Export meshed model
            export_start_time = time.time()
            export_meshed_file(output_file, EXPORT_CONFIG)
            export_time = time.time() - export_start_time

            print(f"Model export completed successfully in {export_time:.2f}
    seconds")
            print(f"Output file: {output_file}")

            # Record successful processing
            successful_meshes.append(mesh_size)

            # Calculate total time for this mesh size
            total_mesh_time = time.time() - mesh_start_time

```

```
            logging.info(f"MESH SIZE {mesh_size} PROCESSING COMPLETED SUCCESSFULLY")
            logging.info(f"- Total processing time: {total_mesh_time:.2f} seconds")
            logging.info(f"- Meshing time: {mesh_process_time:.2f} seconds")
            logging.info(f"- Export time: {export_time:.2f} seconds")
            logging.info(f"- Output file size: {os.path.getsize(output_file):,} bytes")

        except Exception as e:
            # Handle errors
            error_time = time.time() - mesh_start_time

            print(f"ERROR: Failed to process mesh size {mesh_size} mm")
            print(f"Error occurred after {error_time:.2f} seconds")
            print(f"Error details: {str(e)}")

            logging.error(f"MESH SIZE {mesh_size} PROCESSING FAILED")
            logging.error(f"- Processing time before failure: {error_time:.2f} seconds")
            logging.error(f"- Error type: {type(e).name}")
            logging.error(f"- Error message: {str(e)}")
            logging.error(f"- Full error traceback:", exc_info=True)

            failed_meshes.append(mesh_size)

        finally:
            # Cleanup and separator addition
            log_mesh_size_separator(LOGGING_CONFIG['log_file_path'])
            print(f"Mesh size {mesh_size} processing completed.")

    # ======================================================================
    # FINAL PROCESSING SUMMARY AND REPORTING
    # ======================================================================
    total_script_time = time.time() - script_start_time

    # Display comprehensive summary
    print(f"\n{'='*80}")
    print("PROCESSING SUMMARY")
    print(f"{'='*80}")

    if successful_meshes:
        print(f"    Successfully processed mesh sizes: {successful_meshes}")
        logging.info(f"Successfully processed {len(successful_meshes)} mesh sizes: {successful_meshes}")

    if failed_meshes:
        print(f"    Failed mesh sizes: {failed_meshes}")
        logging.info(f"Failed to process {len(failed_meshes)} mesh sizes: {failed_meshes}")

    # Report processing statistics
    success_rate = (len(successful_meshes) / len(TARGET_MESH_SIZES)) * 100
    print(f"\nProcessing Statistics:")
    print(f"- Total mesh sizes attempted: {len(TARGET_MESH_SIZES)}")
    print(f"- Successful: {len(successful_meshes)}")
    print(f"- Failed: {len(failed_meshes)}")
    print(f"- Success rate: {success_rate:.1f}%")
```

```
    print(f"- Total execution time: {total_script_time:.2f} seconds ({total_script_time/60:.1f} minutes)")

    # Log final statistics
    logging.info("="*120)
    logging.info("FINAL PROCESSING SUMMARY")
    logging.info("="*120)
    logging.info(f"Total mesh sizes attempted: {len(TARGET_MESH_SIZES)}")
    logging.info(f"Successfully processed: {len(successful_meshes)} mesh sizes")
    logging.info(f"Failed processing: {len(failed_meshes)} mesh sizes")
    logging.info(f"Success rate: {success_rate:.1f}%")
    logging.info(f"Total script execution time: {total_script_time:.2f} seconds ({total_script_time/60:.1f} minutes)")

    if successful_meshes:
        logging.info(f"Successful mesh sizes: {successful_meshes}")
        for mesh_size in successful_meshes:
            output_file = generate_output_filename(PATHS_CONFIG['output_paths'], mesh_size)
            logging.info(f"  - Mesh size {mesh_size} mm: {output_file}")

    if failed_meshes:
        logging.info(f"Failed mesh sizes: {failed_meshes}")
        logging.info("Review error messages above for troubleshooting guidance")

    # Performance metrics
    avg_time_per_mesh = total_script_time / len(TARGET_MESH_SIZES)
    logging.info(f"Average processing time per mesh size: {avg_time_per_mesh:.2f} seconds")

    if successful_meshes:
        logging.info("RECOMMENDATIONS FOR NEXT STEPS:")
        logging.info("1. Review mesh quality in ANSA for each successful mesh size")
        logging.info("2. Compare element counts and quality metrics across mesh sizes")
        logging.info("3. Proceed with finite element analysis using the generated files")
        logging.info("4. Consider mesh convergence study using the different mesh densities")

    if failed_meshes:
        logging.info("TROUBLESHOOTING RECOMMENDATIONS:")
        logging.info("1. Check geometry quality and validity in the STEP file")
        logging.info("2. Verify property IDs exist in the model")
        logging.info("3. Consider adjusting mesh quality parameters for complex geometries")
        logging.info("4. Increase mesh size for failed cases to improve meshing success")

    # Final completion message
    print(f"\nScript execution completed. Check log file for detailed information:")
    print(f"Log file location: {LOGGING_CONFIG['log_file_path']}")

    # Finalize the log file
    finalize_log(LOGGING_CONFIG['log_file_path'])

    # Display final status
```

```
549     if len(successful_meshes) == len(TARGET_MESH_SIZES):
550         print("\  n      ALL MESH SIZES PROCESSED SUCCESSFULLY!")
551         logging.info("ALL MESH SIZES PROCESSED SUCCESSFULLY - SCRIPT COMPLETED
    WITHOUT ERRORS")
552     elif len(successful_meshes) > 0:
553         print(f"\ n    PARTIAL SUCCESS: {len(successful_meshes)}/{len(
    TARGET_MESH_SIZES)} mesh sizes completed successfully")
554         logging.info(f"PARTIAL SUCCESS - {len(successful_meshes)}/{len(
    TARGET_MESH_SIZES)} mesh sizes completed")
555     else:
556         print("\ n   ALL MESH PROCESSING FAILED - CHECK LOG FILE FOR DETAILS")
557         logging.info("CRITICAL: ALL MESH PROCESSING OPERATIONS FAILED")
558 
559     print(f"\nTotal execution time: {total_script_time/60:.1f} minutes")
560     print("="*80)s
```

Listing 8.1: Phase 1 Meshing Automation Script